

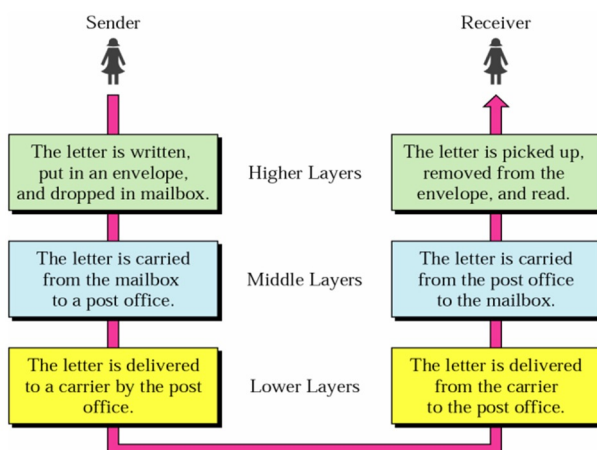


1



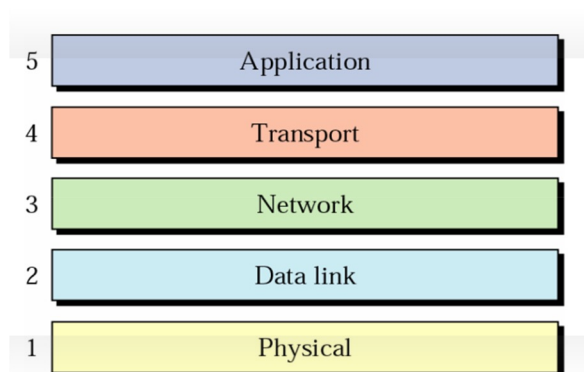
2

Trao đổi thông tin trên thực tế



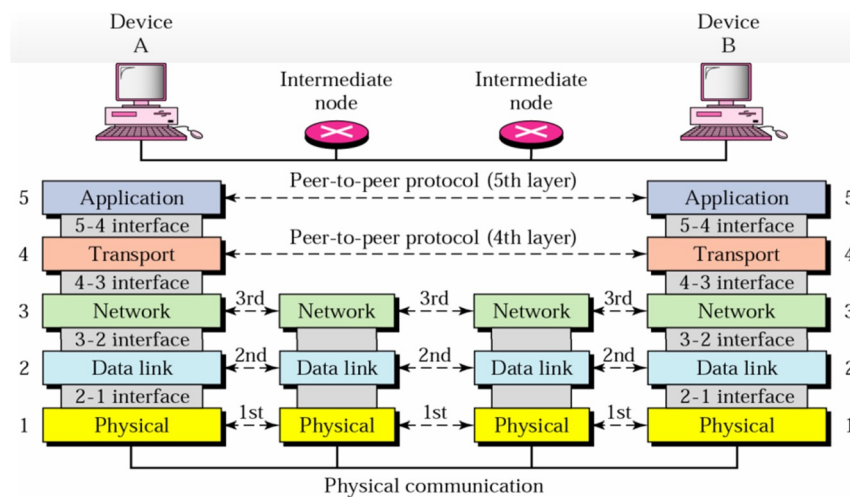
3

Các lớp giao thức Internet



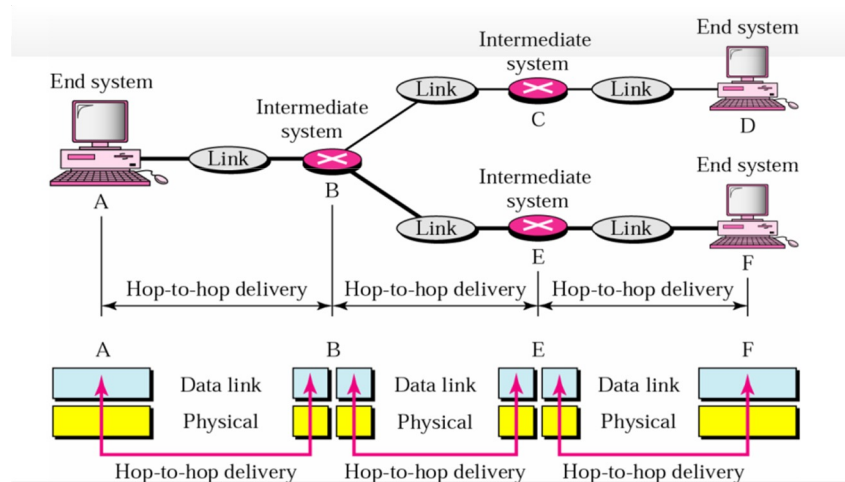
4

Giao tiếp trên hệ thống truyền thông



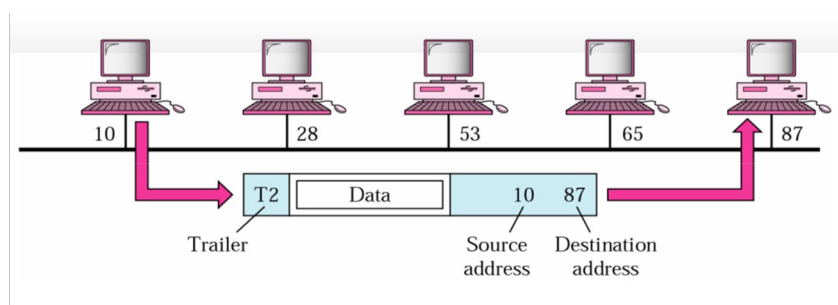
5

Kết nối qua từng chặng



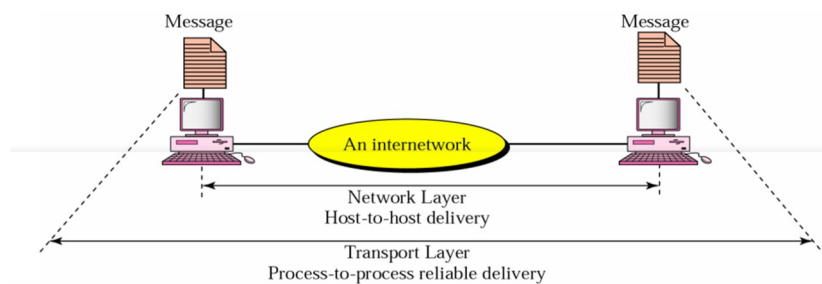
6

Giao tiếp trên mỗi chặng



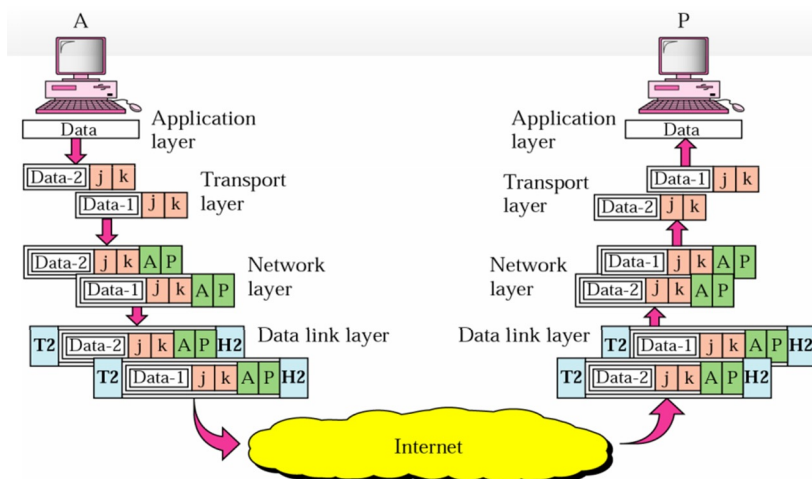
7

Chuyển phát tin cậy một thông điệp



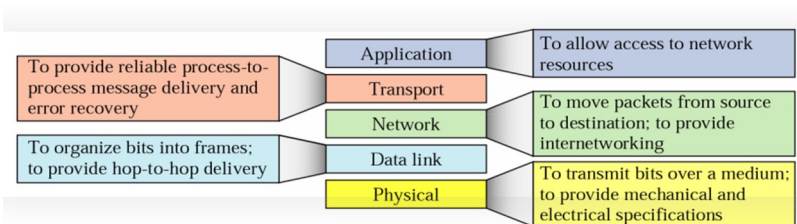
8

Ví dụ



9

Tóm tắt nhiệm vụ của từng lớp giao thức



10

Hai chế độ hoạt động của giao thức

- **Connection-oriented (Connection-based)**
 - Giao tiếp ngang hàng có duy trì một đường đi cố định
- **Connectionless (Datagram)**
 - Giao tiếp ngang hàng không có đường nối cố định
 - Mỗi gói tin có thể đi trên các con đường khác nhau đến đích

11

Tổng quan lập trình mạng (1/2)

- Mạng máy tính có hai phần
 - Vật lý (phần cứng)
 - Luận lý (phần mềm)
- Giao tiếp vật lý
 - Chuẩn giao tiếp vật lý
 - Chip logic có thể lập trình
 - Chương trình điều khiển (driver hay có sẵn trong BIOS, trong hệ điều hành)
 - Lập trình: các driver, các **API**

12

Tổng quan lập trình mạng (2/2)

- Giao tiếp luận lý
 - Giao thức mức thấp: Đặc tả giao thức trong các RFC □ lập trình giao thức
 - Giao thức mức cao: Đặc tả giao thức trong các RFC □ lập trình giao thức.
- **Ứng dụng mạng**: Các chương trình tiện ích của người dùng trao đổi thông tin hay sử dụng dịch vụ qua mạng □ lập trình dựa vào API

13

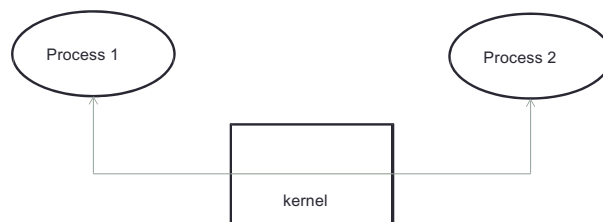
Giao tiếp liên tiến trình

- Process
- Hệ thống *đa nhiệm*
- *Cơ chế cho phép giao tiếp giữa các process trong hệ thống đa nhiệm*: IPC (Inter-Process Communication)

14

Hai loại giao tiếp IPC

- Các process trên cùng một máy

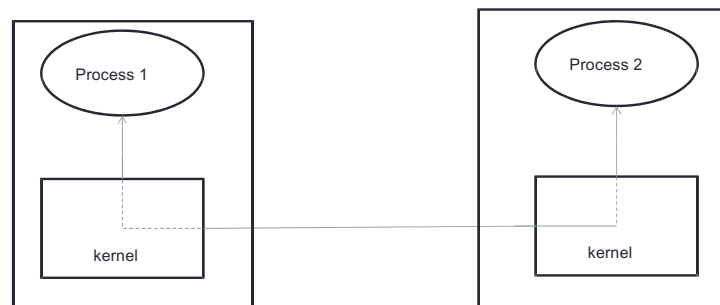


15

Hai loại giao tiếp IPC

- Hai tiến trình trên hai máy khác nhau

*Các kernel thỏa thuận với nhau các quy tắc trao đổi thông tin.
Các quy tắc là một phần của giao thức*



16

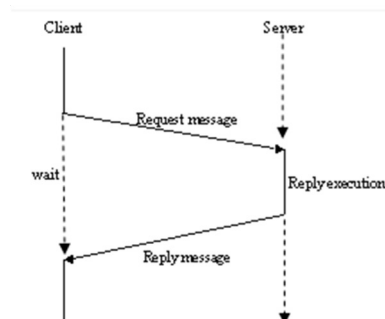
Ứng dụng mạng

- Chương trình tạo ra các tiện ích (dịch vụ) phục vụ người dùng
 - Tiện ích về lưu trữ và truy xuất tập tin (ftp)
 - Tiện ích đầu cuối từ xa (telnet)
 - Tiện ích in ấn
 - Tiện ích duyệt web
- Ứng dụng mạng được xây dựng dựa vào mô hình ứng dụng : **peer-to-peer**, **client-server**, **hybrid** (lai ghép).

17

Mô hình client-server

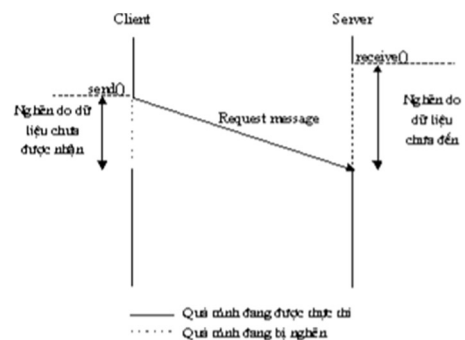
- Ứng dụng cấu thành từ hai phần:
 - Quá trình chuyên cung ứng sự phục vụ là server
 - Quá trình chuyên yêu cầu phục vụ từ server gọi là client



18

Hai chế độ giao tiếp

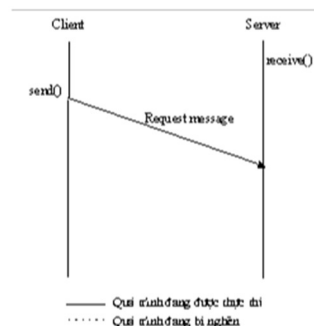
- **Chế độ blocking** (khóa/nghe): khi client hay server phát ra lệnh send gửi thông điệp, quá trình client hay server tương ứng sẽ bị treo cho đến khi phía nhận phát lệnh receive tiếp nhận thông điệp. Tương tự khi gọi lệnh receive mà chưa có send cũng sẽ vào trạng thái treo cho đến khi có lệnh send từ xa gửi thông điệp đến.



19

Hai chế độ giao tiếp (tt)

- **Chế độ nonblocking**: khi phát ra lệnh truyền thông, quá trình client hay server vẫn tiếp tục được thực thi mà không bị treo



20

Kiến trúc chương trình

- Các chức năng trong một chương trình ứng dụng gồm :
 - Giao diện nhập liệu (user interface service)
 - Tính toán, xử lý thông tin theo qui định (business rule service)
 - Truy vấn và lưu trữ thông tin (data storage service)
- Ở mức vật lý các chức năng này được lập trình trong một hay nhiều tập tin tùy theo kiểu kiến trúc :
 - Kiến trúc một tầng
 - Kiến trúc hai tầng
 - Kiến trúc ba tầng

21

Kiến trúc một tầng (single-tier architecture)

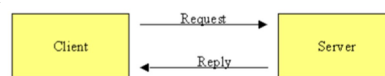
- Cả ba chức năng của phần mềm ứng dụng đều được lập trình trong cùng một tập tin



22

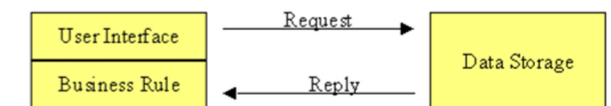
Kiến trúc hai tầng (two-tier architecture)

- Được gọi là kiến trúc client server
- Gồm 2 chương trình thực thi
- Trên hai máy tính khác nhau hay trên cùng một máy tính
- Mỗi ứng dụng phải có giao thức (protocol) được định nghĩa để giao tiếp trao đổi thông tin

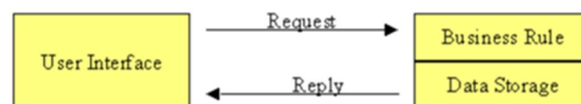


- Thông thường client đảm nhận user interface
- Server đảm nhận data storage
- Chức năng business rule có thể
 - Lập trình tạo ra trên client □ Fat client
 - Lập trình tạo ra trên server □ Fat server

23



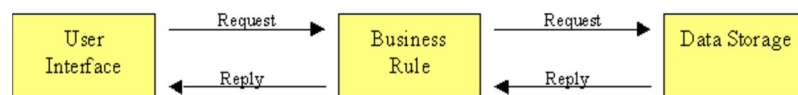
Fat Client



Fat Server

24

Kiến trúc 3 tầng



25

Nhắc lại, NGÔN NGỮ JAVA

26

26

Đặc điểm

- Ngôn ngữ cấp cao
- Thuần hướng đối tượng
- Có thể thực thi trên các hệ điều hành khác nhau
- Đa luồng
- Phân tán
- Ngôn ngữ động
- Đơn giản, dễ học, kiến trúc chương trình dễ hiểu

27

Máy ảo java (JMV)

- Đảm bảo cho chương trình chạy trên nhiều nền tảng khác nhau.
- ByteCode là ngôn ngữ máy của máy ảo giống các mã lệnh nhị phân của máy thật.
- Một chương trình được viết được lưu thành tập tin có phần mở rộng là .java, phải được biên dịch thành tập tin thực thi trên máy ảo java có phần mở rộng là .class.
- Tập tin thực thi chứa các chỉ thị ở dạng ByteCode mà máy ảo java hiểu được.
- Khi thực thi chương trình, máy ảo dịch các ByteCode thành mã nhị phân của máy tính thật
- => máy ảo là trình biên dịch

28

Hai kiểu ứng dụng

- Applet
 - Chương trình nhúng vào trang web, được tải từ webserver về máy client
- Application
 - Chương trình thực thi trực tiếp trên máy ảo java

29

Bộ công cụ phát triển ứng dụng JDK

- Được cung cấp miễn phí bởi JavaSoft (Sun), javasoft.com
- Mỗi loại hệ điều hành sẽ có bộ JDK tương ứng
- Các chương trình thực thi quan trọng của JDK
 - **javac**: biên dịch chương trình nguồn thành tập tin thực thi trên máy ảo
 - **java**: làm cho máy ảo biên dịch mã ByteCode thành mã thực thi trên máy thật
 - **appletviewer**: thông dịch, thực thi các chương trình applet
 - **jdb**: trình gỡ rối
 - **javadoc**: tự động tạo tài liệu chú thích chương trình nguồn
 - **rmic**: tạo Stub cho ứng dụng kiểu RMI
 - **rmiregistry**: thực hiện danh bạ trong RMI

30

Cài đặt hệ thống

- Download JRE và JDK từ javasoft.com phù hợp với hệ điều hành
- Cài đặt, chọn nơi cài đặt ví dụ C:\jdk1.6.0_33
- Thiết lập biến môi trường
 - PATH = C:\jdk1.6.0_33\bin;
 - CLASSPATH = C:\jdk1.4\lib;.;

31

31

Kiểu dữ liệu

- Số:

Tên kiểu	Kích thước
byte	1 byte
short	2 bytes
int	4 bytes
long	8 bytes
float	4 bytes
double	8 bytes
- Ký tự char
 - 2byte
- Kiểu chuỗi String
 - Là lớp thuộc thư viện chuẩn của java, java.lang.String
- Kiểu logic boolean: 2 giá trị true và false
- Kiểu mảng
 - khai báo int[] a; hay int a[]; int maTran[][]
 - Khởi tạo biến a= new int[10]; maTran=int[9][9]
 - Sử dụng mảng int i=a[10]; int x=maTran[2][5]

32

Các phép toán cơ bản

Tương tự như C++

- Toán số học: +, -, *, /, %, =, ++, --, +=, -=, /=, %=
- Toán logic: ==, !=, &&, ||, !, >, <, >=, <=
- Toán trên bit: &, |, ^, <<, >>, ~
- Điều kiện: ?:
- Chuyển đổi kiểu (ép kiểu): (kiểu mới)

33

Nguyên tắc đặt tên

- Tên phân biệt chữ hoa và chữ thường
- Dùng chữ cái, chữ số, ký tự _ và dấu \$
- Không bắt đầu bằng chữ số

Tên lớp: Viết hoa các ký tự đầu của

từ Ví dụ: InputStream, WhileDemo

Tên biến, tên hằng hay tên phương thức: từ đầu tiên viết thường, các từ tiếp theo viết hoa ký tự đầu

Ví dụ maLop, giaTriDau, getInputStream(), maLop,

34

Chương trình ứng dụng kiểu Application

- Ngôn ngữ thuần đối tượng
- Một chương trình ứng dụng chỉ có một lớp thực thi
- Lớp thực thi phải có tên trùng với tập tin chứa nó, phải khai báo public và chứa phương thức:

```
public static void main (String[] args) {
    .....
}
```

Là phương thức được thực thi đầu tiên

35

Simple program

```
App.java
// Chương trình thuộc package nào (chỉ thuộc 1 package)
package package_name;

// Import class vào để sử dụng (có thể import nhiều class)
import java.util.Scanner; // Import một class
import java.util.*; // Import toàn bộ package

// Định nghĩa thêm các class khác
class Util {
    ...
}

// Class chính, là public class duy nhất trong file
public class App {
    ...
    // Class chính phải có method main() như sau
    public static void main(String[] args) {
        ...
    }
}
```

36

36

```

App.java
// Biến không khai báo ở đây
public class App {
    // Biến thực thể
    int instanceVar = 10;

    // Biến tĩnh
    static double staticVar; // Không cần init value

    // Đây là method
    public static void main() {
        // Biến cục bộ
        int localVar = "Hello", name = "John";
    }
}

```

37

37

Lập trình

- HelloWorld
- Dùng JDK để biên dịch và thực thi

```

J HelloWorld.java > ...
1 public class HelloWorld {
    Run | Debug | Run main | Debug main
2     public static void main(String[] args) {
3         System.out.println(x: "Hello, Java!");
4     }
5 }

```

```

Cmd:
javac HelloWorld.java // tạo file class

java HelloWorld //chạy chtr từ .class file
Hoặc
java HelloWorld.java //chạy chtr trực tiếp từ .java file

```

38

Ví dụ in ra màn hình

Dùng

System.out.print (tham số 1+ tham số 2 + tham số 3 + ...)

```
public class HelloJava {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        System.out.println(x: "Hello, Java!");

        int i = 2151569;
        String str = "Phát triển hệ thống tích hợp";

        System.out.println("Mã " + i + " là mã của môn học " + str);
    }
}
```

39

Ví dụ nhập ký tự từ bàn phím

Dùng System.in.read();

```
import java.io.*;
public class KeyRead {
    public static void main(String args[]) {
        try {
            int ch = System.in.read();
            System.out.print("Ký tự " + (char)ch + "
có mã ascii = "+ch);
        } catch (IOException ie) {
            System.out.print("Error " + ie) ;
        }
    }
}
```

40

Cấu trúc if

Cú pháp:

```
if (Condition) {
    // Các lệnh sẽ được thực hiện nếu giá
    trị của Condition là true
}
if (Condition) {
    // Các lệnh sẽ được thực hiện nếu giá trị của
    Condition là true
} else {
    // Các lệnh sẽ được thực hiện nếu giá trị của
    Condition là false
}
```

41

Ví dụ lệnh if

```
//If demo
System.out.print(s: "Vui long nhap mot ky tu:");
try {
    int ch = System.in.read();
    if (ch == 'A'){
        System.out.print(s: "Ban rat may man !");
    }
    else {
        System.out.print(s: "Ban khong gap may !");
    }
} catch(IOException ie) {
    System.out.print("Error:"+ie);
}
```

42

Lệnh while

Cú pháp

```
while (condition) {
    // nếu condition có giá trị là true, thì các tác vụ ở
    // đây sẽ được lặp lại
}
```

Ví dụ

```
import java.io.*;
public class WhileDemo {
    public static void main(String
        args[]) { int num = '9';
        while (num > '0') {
            System.out.print((char)num + "
            "); num--;
        }
    }
}
```

43

Lệnh do-while

Cú pháp

```
do {
    // Lặp lại các tác vụ ở đây cho đến khi điều kiện
    // condition có giá trị là false
} while (condition)
```

Ví dụ

```
import java.io.*;
public class DoWhileDemo {
    public static void
        main(String args[]) { int
        num = '9';
        do {
            System.out.print((char)num
            + " "); num--;
        } while (num > '0');
    }
}
```

44

Lệnh for

Cú pháp

```
for (operation1; condition; operation2){
    // Các tác vụ được lặp lại
}
```

Tương đương như cấu trúc sau: operation1;
while (condition) {
 // Các tác vụ được lặp lại
 operation2;
}

45

Ví dụ lệnh for

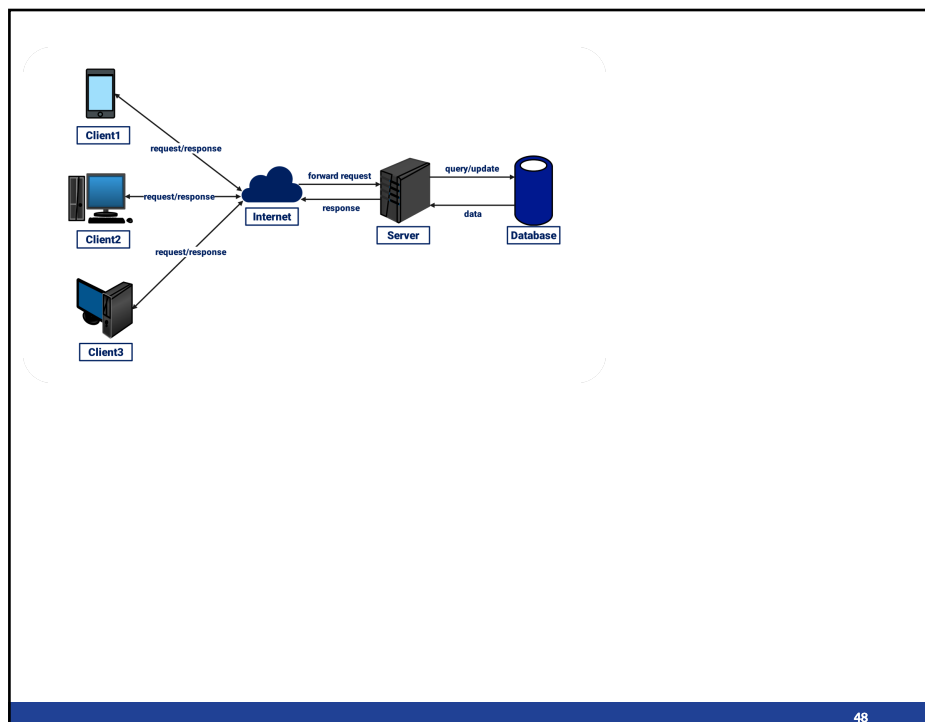
```
import java.io.*; public
class ForDemo    {
    public static main(String args[])
    {
        void
        for(int num = '9'; num>'0'; num--)
            System.out.print((char)num + " ");
    }
}
```

46

Bài tập

- Tìm một ứng dụng mạng bất kỳ và mô tả :
 - Các chức năng hỗ trợ của ứng dụng
 - Kiến trúc của ứng dụng
 - Giao thức hỗ trợ ứng dụng
 - Xác định các trạng thái của các máy và sơ đồ hoạt động chuyển đổi giữa các trạng thái (Chu trình hoạt động của mỗi máy liên quan trong ứng dụng mà qua đó xác định giải thuật của chương trình)

47



48

Lập trình hướng đối tượng (OOP) trong Java

- Là phương pháp thiết kế chương trình dựa trên các Lớp (Class) và Đối tượng (Object),
- tập trung vào việc tổ chức mã nguồn thành các thực thể thực tế với thuộc tính (dữ liệu) và phương thức (hành động), giúp code dễ bảo trì, tái sử dụng và mở rộng nhờ 4 tính chất cốt lõi (Pillars of OOP):
 1. *Đóng gói (Encapsulation),*
 2. *Kế thừa (Inheritance),*
 3. *Đa hình (Polymorphism), và*
 4. *Trừu tượng (Abstraction).*

49

49

Các khái niệm cơ bản

- **Đối tượng (Object):** Một thể hiện (instance) của một lớp, có trạng thái (thuộc tính) và hành vi (phương thức) riêng.
 - Ví dụ: một đối tượng SinhVien có tên, mã số (thuộc tính) và có thể thi (phương thức).
- **Lớp (Class):** Một bản thiết kế, khuôn mẫu để tạo ra các đối tượng. Định nghĩa các thuộc tính và phương thức chung cho các đối tượng thuộc lớp đó.

50

50

Tính đóng gói (Encapsulation)

- Dữ liệu không cho truy cập trực tiếp
- Chỉ thao tác thông qua **getter / setter**
- Kiểm soát tính hợp lệ của dữ liệu

```
Public Student (String name, int age){
    this.name = name;
    this.age = age;
}
```

student.age = -5

```
public class Student {
    // Thuộc tính để private → không cho truy cập trực tiếp
    private String name;
    private int age;

    // Constructor
    public Student(String name, int age) {
        this.name = name;
        setAge(age); // kiểm soát dữ liệu qua setter
    }

    // Getter
    public String getName() {
        return name;
    }

    // Setter có kiểm tra điều kiện
    public void setAge(int age) {
        if (age > 0) {
            this.age = age;
        } else {
            System.out.println("Tuổi không hợp lệ!");
        }
    }

    public int getAge() {
        return age;
    }
}
```

51

51

Tính kế thừa (Inheritance)

- Lớp con kế thừa lại code của lớp cha
- Tránh viết lại code giống nhau - tái sử dụng code

```
public class Main {
    public static void main(String[] args) {
        // Tạo đối tượng Dog
        Dog dog = new Dog("Lucky");

        // Gọi phương thức kế thừa từ Animal
        dog.eat();

        // Gọi phương thức riêng của Dog
        dog.bark();
    }
}
```

```
// Lớp cha
public class Animal {
    protected String name;

    public void eat() {
        System.out.println("Động vật đang ăn...");
    }
}
```

```
// Lớp con kế thừa Animal
public class Dog extends Animal {

    public Dog(String name) {
        this.name = name;
    }

    public void bark() {
        System.out.println(name + " đang sủa: Gâu gâu!");
    }
}
```

52

52

Đa hình (Polymorphism)

- Cùng 1 lời gọi phương thức
- Nhưng hành vi khác nhau
- Xảy ra khi override phương thức

```
// Lớp cha
public class Animal {
    public void makeSound() {
        System.out.println("Động vật phát ra âm thanh...");
    }
}
```

```
// Lớp con Dog
public class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Gâu gâu!");
    }
}
```

```
// Lớp con Cat
public class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Meo meo!");
    }
}
```

```
Dog myDog = new Dog();
Cat myCat = new Cat();
```

```
public class Main {
    public static void main(String[] args) {

        Animal a1 = new Dog();
        Animal a2 = new Cat();

        a1.makeSound(); // Gâu gâu
        a2.makeSound(); // Meo meo
    }
}
```

53

53

Trừu tượng (Abstraction)

- Không quan tâm làm thế nào
- Chỉ quan tâm làm cái gì ✓
- Dùng abstract class hoặc interface

54

54

● Cách 1: Abstract class

vehicle-demo/
 └─ Vehicle.java → *abstract class*
 └─ Car.java → *extends class*
 └─ Bike.java → *extends class*
 └─ Main.java

```
public class Car extends Vehicle {
    @Override
    public void move() {
        System.out.println("Xe hơi chạy bằng 4 bánh");
    }
}
```

```
public abstract class Vehicle {

    // Phương thức trừu tượng
    public abstract void move();

    // Phương thức thường
    public void start() {
        System.out.println("Phương tiện bắt đầu chạy");
    }
}
```

```
public class Bike extends Vehicle {
    @Override
    public void move() {
        System.out.println("Xe máy chạy bằng 2 bánh");
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Vehicle v1 = new Car();
        Vehicle v2 = new Bike();

        v1.move();
        v2.move();
    }
}
```

// abstract class không thể tạo đối tượng trực tiếp → phải tạo đối tượng của lớp con

55

55

● Cách 2: Interface

payment-demo/
 └─ Payment.java → *interface*
 └─ CashPayment.java → *class implements*
 └─ OnlinePayment.java → *class implements*
 └─ CardPayment.java → *class implements*
 └─ Main.java

56

56

Khi nào nên dùng **abstract class** và khi nào nên dùng **interface**?

👉 Dùng abstract class khi:

1. Các lớp con có mối quan hệ "is-a" chặt chẽ
(ví dụ: Dog là một Animal, Car là một Vehicle)
2. Cần chia sẻ code cho các lớp con
 - Có thuộc tính chung
 - Có phương thức đã cài đặt sẵn
3. Muốn kết hợp giữa trừu tượng và cụ thể
 - Có phương thức abstract
 - Có phương thức bình thường
4. Không cần kế thừa từ nhiều lớp khác
(vì Java chỉ cho kế thừa 1 class)

➡ Phù hợp khi mọi Vehicle đều có speed, start(), move()

57

57

Khi nào nên dùng **abstract class** và khi nào nên dùng **interface**?

👉 Dùng interface khi:

1. Chỉ muốn định nghĩa hành vi (what), không quan tâm cài đặt (how)
2. Cần đa kế thừa
 - Một lớp có thể implements nhiều interface
3. Các lớp không liên quan về mặt phân cấp, nhưng có cùng hành vi
(ví dụ: Car, Robot, Drone đều có thể Moveable)
4. Thiết kế theo hướng mở rộng, linh hoạt, plugin

➡ CashPayment, OnlinePayment, CryptoPayment đều có thể triển khai pay()

58

58

Một số kỹ thuật thông dụng trong LTM

- **STREAM (dòng chảy)**
 - **java.io.InputStream**
 - **java.io.OutputStream**
- PIPE (đường ống lệnh)
 - java.io.PipedInputStream
 - java.io.PipedOutputStream
- **THREAD VÀ MULTI-THREAD**

59

59

Kỹ thuật giao tiếp dùng Stream

- Khái niệm Stream
- Input Stream và Output Stream
- Stream trong java

60

60

Khái niệm Stream

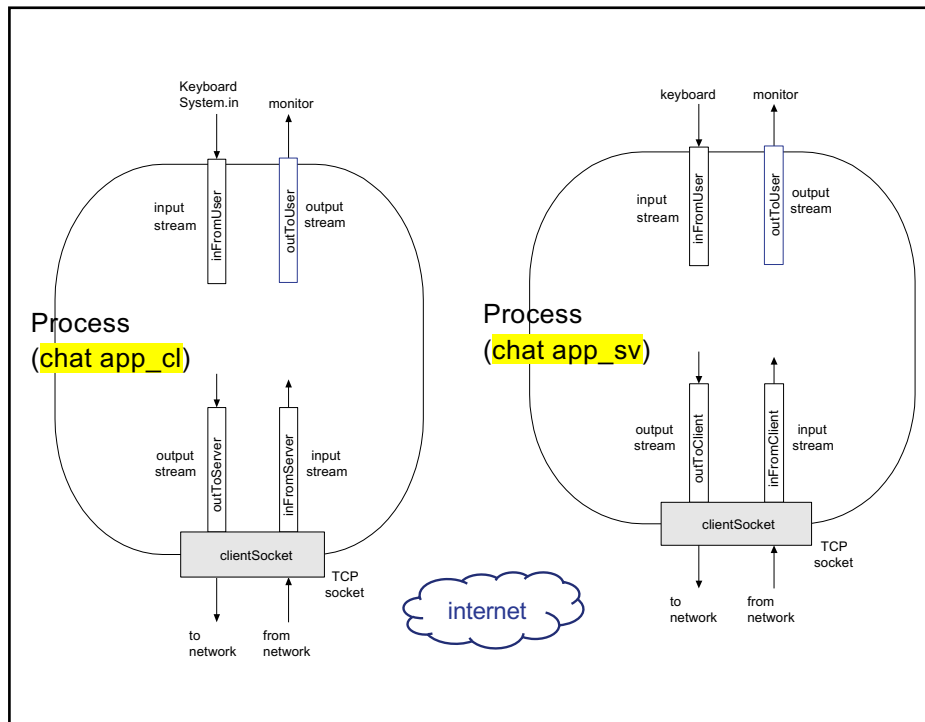
- Stream là dòng liên tục, có **thứ tự của các bytes dữ liệu** *chảy giữa chương trình/ứng dụng và các thiết bị ngoại vi*
- Dùng stream có thể kết nối nhiều thiết bị ngoại vi với chương trình

61

Input Stream và Output Stream

- Nếu dòng dữ liệu trong Stream có hướng chảy từ thiết bị ngoại vi vào chương trình thì ta nói đây là Stream nhập (Input Stream)
- Chảy theo chiều ngược lại là Stream xuất (Output Stream)

62



63

Cách giao tiếp dùng stream

- Để giao tiếp, trước tiên chương trình phải lấy được các stream nhập / xuất gắn với thiết bị.
- Sau đó, chương trình có thể gửi dữ liệu ra ngoại vi bằng thao tác ghi vào Stream xuất của thiết bị.
- Có thể nhận dữ liệu từ ngoại vi bằng thao tác đọc Stream nhập của thiết bị .

64

Stream trong java

- Đối với Java, các thiết bị đều có Stream nhập/Stream xuất nối với nó.
- Java hỗ trợ hai các lớp stream cơ bản trong gói java.io là:
 - `java.io.InputStream`: Stream nhập → `System.in`
 - `java.io.OutputStream`: Stream xuất → `System.out`

65

Lớp java.io.InputStream

- Là loại nhận dữ liệu từ bên ngoài
- Có phương thức:
 - `int read() throws IOException`
 - Đọc 1 byte từ Stream
 - Return 0-255 : Mã ASCII của byte nhận được từ ngoại vi
 - 1 : Stream đã kết thúc, không còn dữ liệu.
- Đối với java `System.in` là một `InputStream` nối với bàn phím

66

Ví dụ 1a

1b: string

```

import java.io.IOException;
import java.io.InputStream;

public class InStream1 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        InputStream is = System.in;
        while(true){
            try {
                int ch = is.read();
                if(ch == -1 || ch == 'q'){
                    break;
                }
                System.out.println((char)ch);
            } catch (IOException ioe) {
                System.out.println(ioe);
            }
        }
    }
}

```

67

67

Lớp java.io.InputStream

int read(byte b[]) throws IOException

Đọc tất cả các byte hiện có trong Stream vào mảng b.

Return 0-255: Số lượng byte đọc được.

-1 : Stream đã kết thúc, không còn dữ liệu.

int read(byte b[], int offset, int len)

Đọc len byte từ Stream hiện tại, lưu vào trong mảng b bắt đầu từ vị trí offset

Return: số lượng byte đọc được.

-1 : Stream đã kết thúc.

int available(): Trả về số lượng byte hiện có trong Stream mà không làm nghẽn chương trình

68

Ví dụ 2

```
import static java.lang.Thread.*;

import java.io.IOException;
import java.io.InputStream;

public class InStream2b {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) throws InterruptedException, IOException {
        InputStream is = System.in;

        try {
            while (true) {
                // Kiểm tra xem có bao nhiêu byte dữ liệu có sẵn mà không bị chặn
                if (is.available() > 0) {
                    byte[] buffer = new byte[is.available()];
                    int bytesRead = is.read(buffer);
                    if (bytesRead == -1) {
                        break; // Kết thúc input stream
                    }
                    String str = new String(buffer, offset:0, bytesRead);
                    System.out.print(str); // In ra dữ liệu đọc được
                } else {
                    // Nếu không có dữ liệu sẵn có, in một ký tự '.' và đợi một chút
                    System.out.print(c:'.');
                    sleep(millis:100); // Ngừng một chút để giảm tải CPU
                }
            }
        } catch (IOException e) {
        }
    }
}
```

69

69

Lớp java.io.OutputStream

- Cho phép chương trình xuất dữ liệu ra thiết bị ngoài có các phương thức:

void write(int b) throws IOException

- Viết byte b vào Stream hiện tại,
- Return : void

void write (byte[] b) throws IOException

- Viết tất cả các phần tử của mảng b vào Stream hiện tại
- Return : void

void write (byte[] b, int offset, int len) throws IOException:

- Viết len phần tử trong mảng b vào Stream hiện tại, bắt đầu từ phần tử có chỉ số là offset của mảng.

- Return : void

System.out là một OutputStream nối với màn hình

70

Nhập chuỗi từ một InputStream ???

! Vấn đề: InputStream chỉ cung cấp phương thức đọc **byte** và **mảng byte**

Giải pháp:

- Lớp `java.io.InputStreamReader`: là cầu nối để chuyển InputStream dạng **byte** sang InputStream dạng **ký tự**
- Lớp `java.io.BufferedReader`: hỗ trợ việc **đọc** văn bản từ một InputStream dạng **ký tự**
- Phương thức **`String readLine()`** throws **`IOException`** của `BufferedReader` cho phép đọc dòng văn bản kế tiếp trong InputStream

71

Ví dụ 3

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class ReadLineDemo {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        InputStream is = System.in;
        InputStreamReader isr = new InputStreamReader(is);
        BufferedReader br = new BufferedReader(isr);
        while (true) {
            try {
                String line = br.readLine();
                if (line == null) break;
                System.out.println(line);
            } catch (IOException e) {
                System.out.println(e);
            }
        }
    }
}
```

72

72

Xuất chuỗi ra một OutputStream

- Vấn đề: tương tự InputStream
- Để có thể gửi được chuỗi ra một OutputStream ta phải sử dụng lớp **java.io.PrintWriter**
- Phải đổi OutputStream dạng **byte** sang dạng **ký tự**, và viết chuỗi vào OutputStream dạng **ký tự**
- Ví dụ in chuỗi ra màn hình

73

Ví dụ 4: in chuỗi ra màn hình

```
import java.io.*;

public class PrintString {
    public static void main(String args[]) {
        OutputStream os = System.out;
        PrintWriter pw = new PrintWriter(os);
        pw.write("This is a string \r\n");    //os.write("hello");
        pw.println("This is a line");        //os.println()
        pw.write("Bye! Bye!");
        pw.flush();
    }
}
```

74

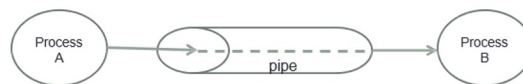
Kỹ thuật ống dẫn (Pipe)

- Giới thiệu
- Các loại ống dẫn
- Ống dẫn trong java

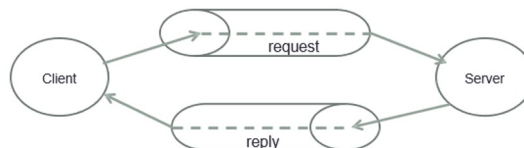
75

Khái niệm về ống dẫn

- Là kỹ thuật được hỗ trợ trong hầu hết các ngôn ngữ lập trình. Tạo cầu nối giao tiếp giữa các tiến trình (process)
- Cho phép 2 tiến trình trên cùng một máy có thể trao đổi dữ liệu với nhau.
- Dữ liệu đi trên ống dẫn theo một chiều nhất định. Một đầu để ghi dữ liệu vào và một đầu để đọc dữ liệu ra



- Ống dẫn cũng có thể áp dụng cho giao tiếp giữa client và server trong ứng dụng dạng client/server bằng cách sử dụng 2 ống dẫn.



76

76

Phân loại ống dẫn

- **Named Pipe** (Ống dẫn có tên)

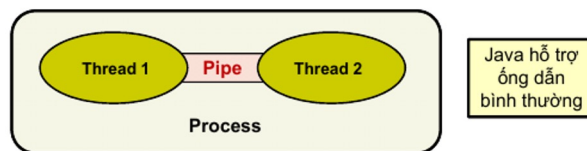
Cho phép hai tiến trình có không gian địa chỉ khác nhau (nhưng phải trên cùng một máy) giao tiếp với nhau.

Giống như 1 tập tin: Dữ liệu sẽ được lấy ra ở đầu tập tin và được thêm vào ở cuối tập tin.

- **Normal Pipe** (Ống dẫn bình thường)

Giới hạn trong phạm vi không gian địa chỉ của 1 tiến trình.

Giao tiếp giữa các thread trong 1 tiến trình



77

77

Ống dẫn trong Java

- Java dùng normal pipe, cho phép giao tiếp giữa các thread qua pipe
- Java hỗ trợ tiện ích ống dẫn qua hai lớp
 - **java.io.PipedInputStream**: kế thừa từ lớp **InputStream**
 - **java.io.PipedOutputStream**: kế thừa từ lớp **OutputStream**

- Các bước tạo ống dẫn:

1. Tạo đầu đọc

PipedInputStream daudoc = new **PipedInputStream**();

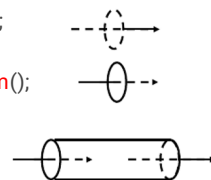
2. Tạo đầu ghi

PipedOutputStream daughi = new **PipedOutputStream**();

3. Nối đầu đọc với đầu ghi hoặc ngược lại

daudoc.**connect**(daughi);

hoặc: daughi.**connect**(daudoc);



Chú ý: các phương thức khởi tạo ở trên đều phải bắt ngoại lệ

78

78

Ống dẫn trong Java

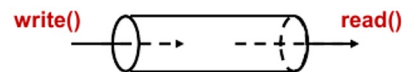
- Sử dụng Pipe trong Java

java.io.PipedInputStream: kế thừa từ lớp InputStream

- Có 3 phương thức read(): đọc dữ liệu từ pipe
- Phương thức close(): đóng pipe và giải phóng tài nguyên

java.io.PipedOutputStream: kế thừa từ lớp OutputStream

- Có 3 phương thức write(): ghi dữ liệu từ pipe
- Phương thức close(): đóng pipe và giải phóng tài nguyên



79

79

Ví dụ về Ống dẫn trong Java

```
import java.io.IOException;
import java.io.PipedInputStream;
import java.io.PipedOutputStream;

public class PipeExample {
    Run | Debug
    public static void main(String[] args) {
        PipedInputStream pis = new PipedInputStream();
        PipedOutputStream pos = new PipedOutputStream();

        try {
            pis.connect(pos);

            pos.write(65);
            pos.write(66);
            pos.write(67);

            System.out.println("Use of available(): " + pis.available());
        } catch (IOException e) {
            // TODO: Auto-generated catch block
            e.printStackTrace();
        }
    }
}
```

80

80

Ví dụ về Ống dẫn trong Java

- Pipe trong java IO cung cấp khả năng giao tiếp giữa giữa 2 luồng (thread) trên cùng một JVM



- Giả sử bạn có 2 luồng, A là luồng dữ liệu đầu vào và B là luồng dữ liệu đầu ra. Hai luồng này chạy trên 2 thread khác nhau vậy làm sao để kết nối A và B lại với nhau?

81

81

Ví dụ về Ống dẫn trong Java

82

82

Ví dụ về Ống dẫn trong Java

Dịch vụ phản hồi thông tin (Echo Service)

- Xây dựng theo mô hình Client-Server
- Server
 - Chờ nhận các byte từ Client gửi đến
 - Khi nhận byte nào thì sẽ gửi trả lại đúng byte đó cho Client
- Client
 - Gửi từng ký tự đến Server
 - Nhận kết quả trả về



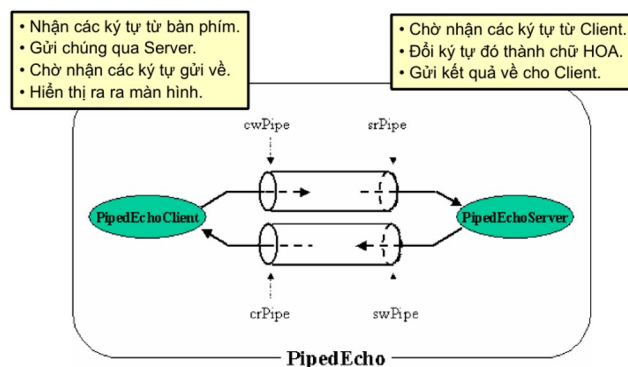
Xây dựng một ứng dụng có tên là PipeEcho mô phỏng dịch vụ phản hồi thông tin sử dụng Pipe làm phương tiện giao tiếp giữa các Thread trong ứng dụng.

83

83

Ví dụ về Ống dẫn trong Java

Mô hình ứng dụng PipeEcho



84

84

Ví dụ về Ống dẫn trong Java

85

85

Thread (Luồng)

- Khái niệm
- Multithread
- Thread trong java

86

86

Khái niệm

- Thread là đơn vị chương trình được dùng trong **cơ chế xử lý song song** nhằm cải thiện hiệu năng của hệ thống máy tính
- Trong hệ điều hành có hỗ trợ tiện ích luồng, đơn vị cơ bản sử dụng CPU là luồng.

Một tiến trình bao gồm một không gian địa chỉ và nhiều luồng điều khiển.

Mỗi luồng có bộ đếm chương trình, trạng thái thanh ghi và ngăn xếp riêng.

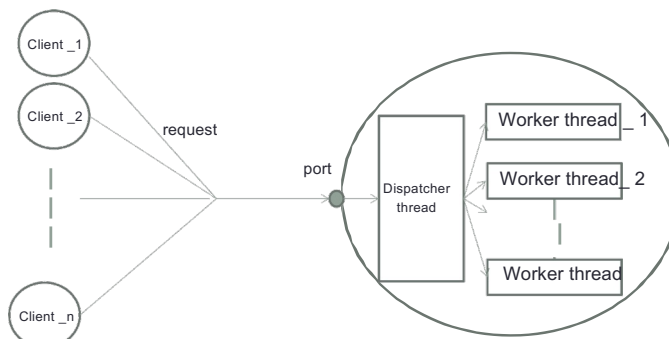
Tất cả các luồng của một tiến trình sẽ cùng nhau chia sẻ một không gian địa chỉ.

87

87

Multi-threads

- Một chương trình có thể mở nhiều thread đồng thời
- Server sử dụng multithread để phục vụ đồng thời nhiều yêu cầu từ client.



88

88

Thread trong java

Thread trong java là đối tượng thuộc lớp `java.lang.Thread`

Cài đặt thread trong chương trình bằng cách **tạo lớp con** của lớp `java.lang.Thread`

89

89

Thread trong java

Ba phương thức cơ bản điều khiển thread

`public native synchronized void start()` : Chuẩn bị mọi thứ để thực hiện

`public void run() {...}` : Thực hiện công việc cụ thể của thread, được kích hoạt một cách tự động bởi phương thức `start()`

`public final void stop()` : Kết thúc luồng

90

90

Ví dụ về Luồng

Định nghĩa lớp `MyThread` là một Thread có:

- Các thuộc tính
 - `name`: tên luồng
 - `n`: số lần luồng xuất hiện ra màn hình
- Các phương thức
 - `MyThread(String name, int n)` : Là phương thức khởi tạo, có nhiệm vụ gán giá trị cho 2 thuộc tính vừa gọi phương thức `start()` để thread hoạt động (***start()* tự động gọi run()**)
 - `run()` : In n lần dòng thông báo ra màn hình rồi kết thúc thread.
 - `main()` : Tạo ra 4 threads thuộc lớp `MyThread` lần lượt có tên là `Thread0`, `Thread1`, `Thread2`, `Thread3`. Mỗi thread có 1000 lần xuất hiện ra màn hình.

91

91

```
package javaThread;

//
public class MyThread extends Thread {
    String name; // ten thread
    int n; // so lan thread xuất hiện ra màn hình

    MyThread(String name, int n) { // khởi tạo, gán giá trị cho 2 thuộc tính
        // gọi start() để cho thread hoạt động (start() tự động gọi run())
        this.name = name;
        this.n = n;
        System.out.println(name + " has been created...");
        start();
    }

    public void run() { // In n dòng thông báo ra màn hình rồi kết thúc thread.
        try {
            for (int i = 0; i < n; i++) {
                System.out.println("Hello, I am " + name);
            }
        } catch (InterruptedException inex) {
            System.out.println("current thread interrupted");
        }
    }

    public static void main(String[] args) {
        int n = 1000;
        int nt = 4;
        for (int i = 0; i < nt; i++) {
            MyThread t = new MyThread("Thread: " + i, n);
        }
    }
}
```

VÍ DỤ 5

93

93

```

PS C:\Users\MinhKhoa\Downloads> java MyThread
Luong - 0vua duoc khoi tao...
Luong - 1vua duoc khoi tao...
Luong - 2vua duoc khoi tao...
Luong - 3vua duoc khoi tao...
Xin chao, tui la luong Luong - 0
Xin chao, tui la luong Luong - 1
Luong - 4vua duoc khoi tao...
Xin chao, tui la luong Luong - 2
Xin chao, tui la luong Luong - 3
Xin chao, tui la luong Luong - 4
Xin chao, tui la luong Luong - 1
Xin chao, tui la luong Luong - 2
Xin chao, tui la luong Luong - 0
Xin chao, tui la luong Luong - 4
Xin chao, tui la luong Luong - 3
Xin chao, tui la luong Luong - 1
Xin chao, tui la luong Luong - 2
Xin chao, tui la luong Luong - 0
Xin chao, tui la luong Luong - 4
Xin chao, tui la luong Luong - 3
Xin chao, tui la luong Luong - 1
Xin chao, tui la luong Luong - 0
Xin chao, tui la luong Luong - 2
Xin chao, tui la luong Luong - 4
Xin chao, tui la luong Luong - 3
Xin chao, tui la luong Luong - 1
Xin chao, tui la luong Luong - 2
Xin chao, tui la luong Luong - 4
Xin chao, tui la luong Luong - 3
Xin chao, tui la luong Luong - 1
Xin chao, tui la luong Luong - 1

Xin chao, tui la luong Luong - 1
Xin chao, tui la luong Luong - 0
Xin chao, tui la luong Luong - 2
Xin chao, tui la luong Luong - 4
Xin chao, tui la luong Luong - 3
Xin chao, tui la luong Luong - 1
Xin chao, tui la luong Luong - 0
Xin chao, tui la luong Luong - 2
Xin chao, tui la luong Luong - 4
Xin chao, tui la luong Luong - 3
Xin chao, tui la luong Luong - 1
Xin chao, tui la luong Luong - 2
Xin chao, tui la luong Luong - 4
Xin chao, tui la luong Luong - 3
Xin chao, tui la luong Luong - 1
Xin chao, tui la luong Luong - 2
Xin chao, tui la luong Luong - 4
Xin chao, tui la luong Luong - 3
PS C:\Users\MinhKhoa\Downloads>

```



Các thread in ra màn hình theo thứ tự không thể xác định được vì chúng được thực thi song song

94

94

Một số phương thức của lớp Thread

- `public static void sleep(long milliseconds) throws InterruptedException` : làm cho thread bị nghẽn (blocked) một khoảng thời gian mili giây xác định.
- `public final void suspend()` : chuyển thread từ trạng thái sẵn sàng sang trạng thái nghẽn.
- `public final void resume()` : chuyển thread từ trạng thái nghẽn sang trạng thái sẵn sàng.
- `public final void yield()` : chuyển thread từ trạng thái đang chạy sang trạng thái sẵn sàng.

95

95

Ưu tiên hóa thread

- Cài đặt mức độ ưu tiên phân phối CPU cho các thread. Khi tất cả các thread đều ở trạng thái ready, thread nào có ưu tiên cao hơn được phân phối CPU trước.
- Độ ưu tiên trong Java được định nghĩa bằng các hằng số nguyên theo thứ tự giảm dần như sau:

`Thread.MAX_PRIORITY`

`Thread.NORM_PRIORITY`

`Thread.MIN_PRIORITY`

96

96

2 Phương thức kiểm soát ưu tiên

- `setPriority( int x )` : Đặt độ ưu tiên của luồng là x
- `int getPriority( )` : Trả về giá trị ưu tiên của luồng

97

97

Ví dụ Ưu tiên hóa luồng

- Trong ví dụ này ta tạo ra 12 threads thuộc lớp MyThread. Một mảng 3 phần tử tên **prio** chứa 3 độ ưu tiên từ cao nhất đến thấp nhất. Thread thứ *i* sẽ có độ ưu tiên ở vị trí *i%3* trong mảng **prio**. Như vậy các thread 0, 3, 6, 9 có độ ưu tiên cao nhất, sau đó đến 1, 4, 7, 10 và cuối cùng là các thread 2, 5, 8, 11.

```
public class PriorityThread{
    Run | Debug
    public static void main(String args[]){
        int n = 10;
        int nt = 12;
        int prio[]={Thread.MAX_PRIORITY, Thread.NORM_PRIORITY, Thread.MIN_PRIORITY};
        for (int i=0; i<nt; i++){
            MyThread t = new MyThread("Thread"+i,n);
            t.setPriority(prio[i%3]); //thread thu i có độ ưu tiên ở vị trí thu i%3
        }
    }
}
```

98

98

```
Hello, I am Thread5
Hello, I am Thread11
    I go to bed now. bye!bye!!
Hello, I am Thread8
    I go to bed now. bye!bye!!
    I go to bed now. bye!bye!!
Hello, I am Thread0
    I go to bed now. bye!bye!!
Hello, I am Thread9
    I go to bed now. bye!bye!!
Hello, I am Thread6
Hello, I am Thread3
    I go to bed now. bye!bye!!
    I go to bed now. bye!bye!!
Hello, I am Thread1
Hello, I am Thread7
    I go to bed now. bye!bye!!
    I go to bed now. bye!bye!!
Hello, I am Thread10
    I go to bed now. bye!bye!!
Hello, I am Thread2
    I go to bed now. bye!bye!!
Hello, I am Thread5
Hello, I am Thread8
Hello, I am Thread11
    I go to bed now. bye!bye!!
Hello, I am Thread4
    I go to bed now. bye!bye!!
    I go to bed now. bye!bye!!
    I go to bed now. bye!bye!!
```



- Các thread 0, 3, 9 có độ ưu tiên cao hơn các thread khác nên chúng được thực thi thường xuyên hơn
- Các thread 2, 5, 8, 11 có độ ưu tiên thấp nhất nên chúng kết thúc sau cùng

99

99

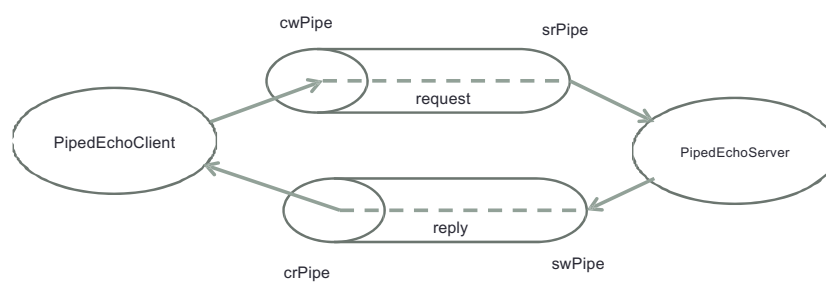
Ví dụ 6 áp dụng

Ứng dụng echo

- Phía Server: Chờ nhận các byte đến từ Client. Với mỗi byte nhận được, Server gửi về đúng byte đã nhận trở lại Client.
- Phía Client: Gửi các byte sang Server và chờ nhận các byte gửi về từ Server.

100

Phân tích thiết kế



101

Phân tích thiết kế

- Client và Server dưới dạng thread
- Dùng hai pipe để giao tiếp
- Chương trình ứng dụng gồm có ba lớp
 - Lớp thực thi phía client
 - Lớp thực thi phía server
 - Lớp thực thi chương trình ứng dụng

102

Bài tập 1

Viết chương trình mô phỏng quá trình "sản xuất - lưu kho- phân phối", trong đó bộ phận sản xuất sẽ sản xuất ra một số lượng ngẫu nhiên n sản phẩm nào đó rồi nhập kho. Bộ phận phân phối yêu cầu xuất kho một số lượng ngẫu nhiên m sản phẩm nào đó từ kho.

Yêu cầu: nhập kho chỉ được chấp nhận nếu số lượng hàng hóa đưa vào không vượt quá sức chứa của kho, nếu không, phải chờ cho đến khi có đủ chỗ trống trong kho. Xuất kho chỉ được chấp nhận khi còn đủ hàng trong kho nếu không cũng phải chờ.

103

Bài tập 2

- Viết chương trình ứng dụng client/server kết nối qua stream
- Client chờ nhập ký tự, gửi sang server, chờ nhận ký tự từ server và in
- Server chờ nhận ký tự từ client, đổi sang dạng chữ hoa rồi gửi lại ch
- Kết quả:** chạy chương trình client: người dùng nhập vào một câu thường rồi nhấn phím enter. Sau đó sẽ thấy hiện lại câu với các từ in

104



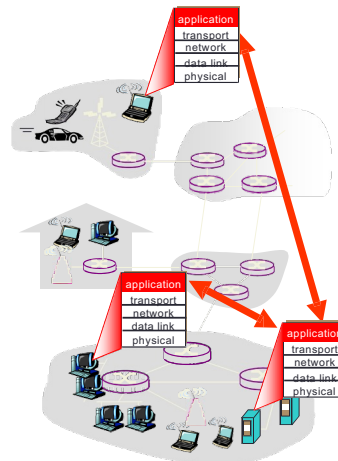
105

Tổng quan

Viết chương trình

- Chạy trên các hệ thống đầu cuối
- Truyền thông qua mạng
- Ví dụ web server giao tiếp với browser

Viết chương trình trên các thiết bị mạng ngoài phạm vi này



106

Mục tiêu

Biết cách xây dựng một ứng dụng client/server giao tiếp bằng socket

Socket API

- Được giới thiệu trong BSD4.1 UNIX, 1981
- Được khởi tạo, sử dụng và hủy một cách tường minh bởi ứng dụng
- Mô hình client/server
- Hai loại dịch vụ truyền tải qua **socket API**:
 - *Connectionless: Datagram không bảo đảm (UDP)*
 - *Connection-oriented bảo đảm (TCP) : 3-ways handshakes*

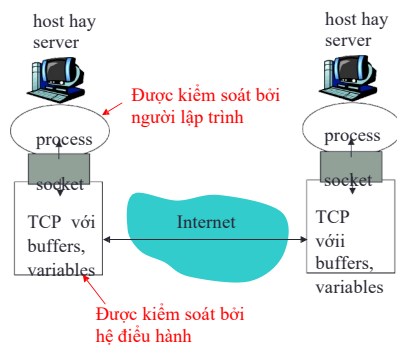
socket

Một **tiếp** cục trên host, được tạo bởi ứng dụng và được điều khiển bởi hệ điều hành, qua đó quá trình ứng dụng có thể truyền (hay nhận) đến (hay từ) quá trình khác

107

Giao tiếp giữa các quá trình trên hai host khác nhau

- process truyền/nhận thông điệp đến/từ **socket** của nó
- socket tương tự như cửa ra vào
 - Quá trình truyền thông điệp ra cửa
 - Quá trình truyền dựa vào hạ tầng truyền tải trên phía kia của cánh cửa, thông tin được chuyển đến socket của quá trình thu
- API: (1) chọn giao thức truyền tải ; (2) khả năng sửa một vài thông số

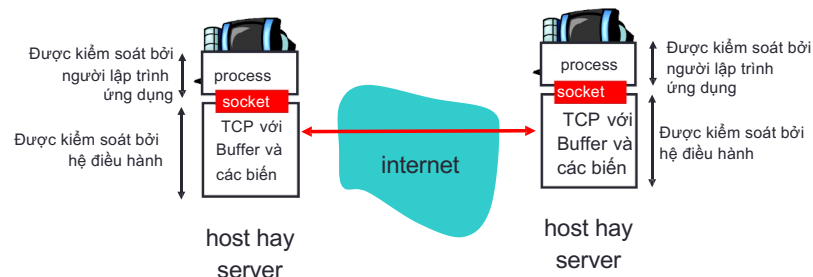


108

Lập trình socket với TCP

Socket: Như là cửa thông giữa các quá trình ứng dụng và giao thức truyền tải end-to-end (UDP hay TCP)

TCP service: truyền tải tin cậy các byte từ một process đến một process khác



109



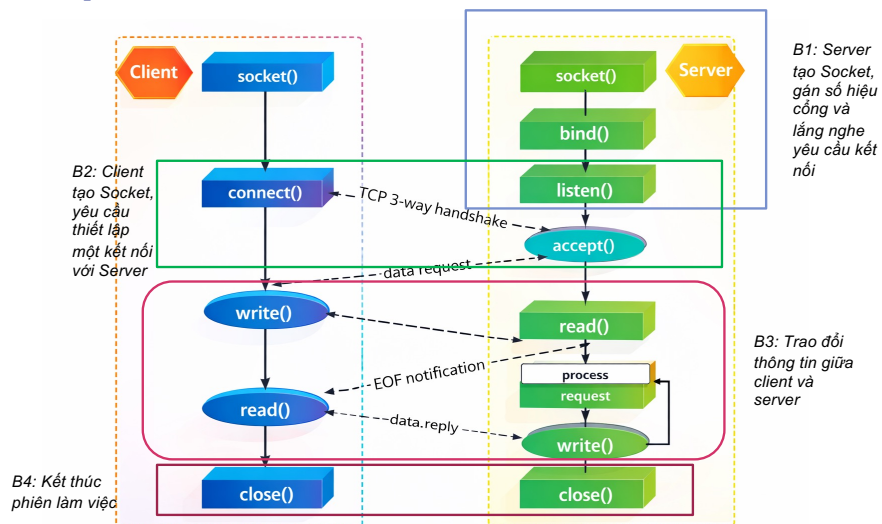
Lập trình socket với TCP

1. **Client phải liên lạc với server**
Trước hết tiến trình trên server phải chạy
server phải tạo socket để đón tiếp client
2. **Client liên lạc server bằng:**
Tạo TCP socket
Chỉ ra **IP address, port number** của quá trình trên server
Khi **client tạo socket**: client TCP thiết lập kết nối đến server TCP
3. Khi được liên hệ bởi client, , **server TCP tạo socket mới** để quá trình server giao tiếp với client
Cho phép server giao tiếp với nhiều quá trình client
Các chỉ số port được dùng để phân biệt các client

110

110

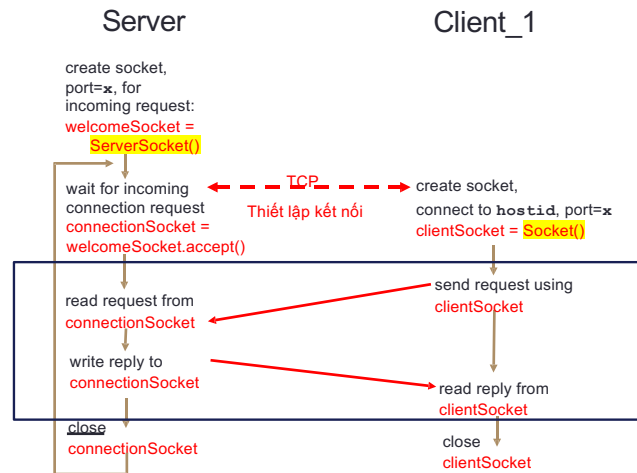
Lập trình socket với TCP



111

111

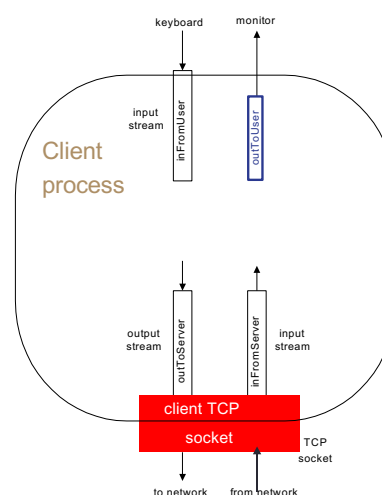
Tương tác giữa client socket và server socket qua TCP



112

Stream với socket

- Một **input stream PHẢI** được kết với socket, SOCKET như là một nguồn nhập của quá trình.
- Một **output stream PHẢI** được kết với socket, SOCKET như là một đích đến của quá trình.



113

Lập trình socket với UDP

- UDP: không có kết nối giữa client và server
- Không bắt tay
 - Sender gán địa chỉ IP và port của đích vào mỗi gói
 - Server phải tách địa chỉ IP và port của client từ gói nhận được
- UDP: data có thể không đúng thứ tự ở máy thu hay mất

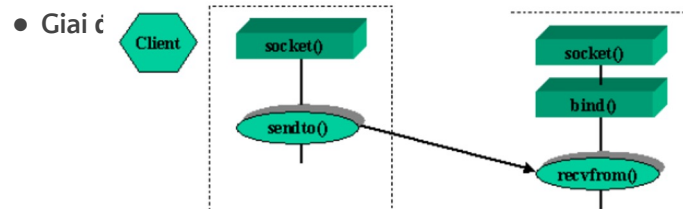
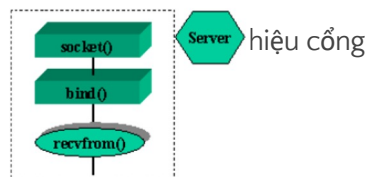
Từ góc độ ứng dụng

UDP cung cấp dịch vụ truyền tải không bảo đảm giữa client và server

114

Lập trình socket với UDP

- Giai đoạn 1: Server

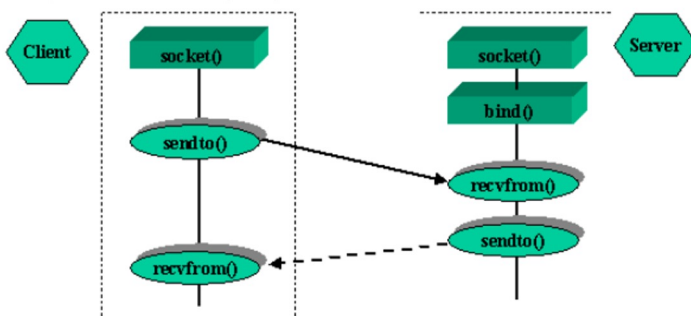


115

115

Lập trình socket với UDP

- Giữa



116

116

Tương tác giữa client socket và server socket qua UDP

Server

Client

create socket,
port= x.
**serverSocket =
DatagramSocket()**

read datagram from
serverSocket

write reply to
serverSocket
specifying
client address,
port number

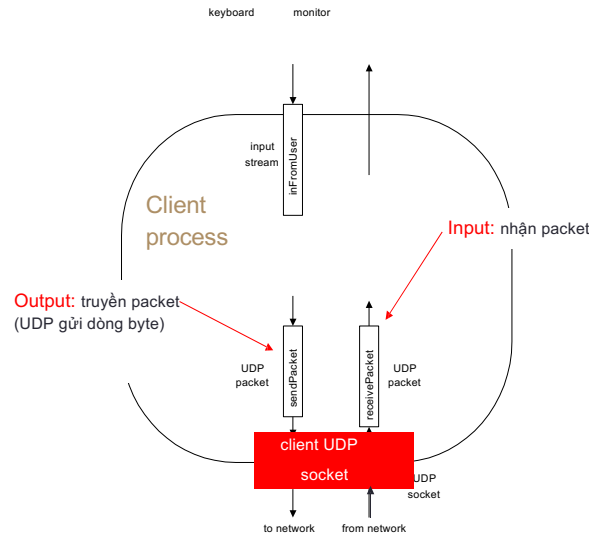
create socket,
**clientSocket =
DatagramSocket()**

Create datagram with server IP and
port=x; send datagram via
clientSocket

read datagram from
clientSocket
close
clientSocket

117

Giao tiếp qua stream



118

Mô hình ứng dụng Client-Server sử dụng Socket ở chế độ có nối kết (TCP) (1/3)

Giai đoạn 1: Server tạo Socket, gán số hiệu cổng và lắng nghe yêu cầu nối kết.

- `socket()`: Server yêu cầu tạo một socket để có thể sử dụng các dịch vụ của tầng vận chuyển.
- `bind()`: Server yêu cầu gán số hiệu cổng (port) cho socket.
- `listen()`: Server lắng nghe các yêu cầu nối kết từ các client trên cổng đã được gán.

Giai đoạn 2: Client tạo Socket, yêu cầu thiết lập một nối kết với Server

- **socket():** Client yêu cầu tạo một socket để có thể sử dụng các dịch vụ của tầng vận chuyển, thông thường hệ thống tự động gán một số hiệu cổng còn rảnh cho socket của Client.
- **connect():** Client gửi yêu cầu nối kết đến server có địa chỉ IP và Port xác định.
- **accept():** Server chấp nhận nối kết của client, khi đó một kênh giao tiếp ảo được hình thành, Client và server có thể trao đổi thông tin với nhau thông qua kênh ảo này.

119

Mô hình ứng dụng Client-Server sử dụng Socket ở chế độ có nối kết (TCP) (2/3)

Giai đoạn 3: Trao đổi thông tin giữa Client và Server.

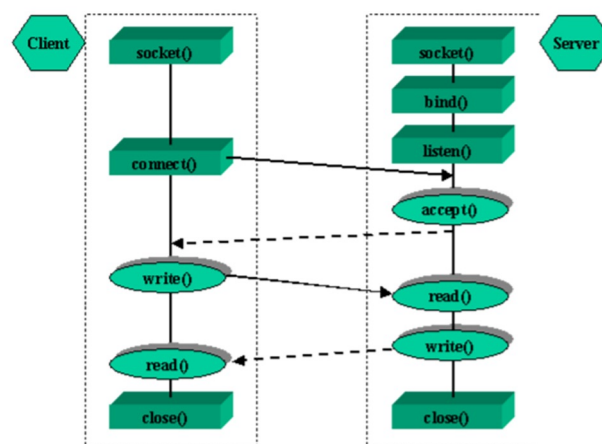
- Sau khi chấp nhận yêu cầu nối kết, thông thường server thực hiện lệnh `read()` và ngẩn cho đến khi có thông điệp yêu cầu (Request Message) từ client gửi đến.
- Server phân tích và thực thi yêu cầu. Kết quả sẽ được gửi về client bằng lệnh `write()`.
- Sau khi gửi yêu cầu bằng lệnh `write()`, client chờ nhận thông điệp kết quả (ReplyMessage) từ server bằng lệnh `read()`.
- Các câu lệnh `read()`, `write()` có thể được thực hiện nhiều lần, tùy theo đặc tả của giao thức truyền thông cụ thể. Các thông điệp request hay reply cũng có định dạng tùy vào giao thức truyền thông

Giai đoạn 4: Kết thúc phiên làm việc

Kênh ảo sẽ bị xóa khi Server hoặc Client đóng socket bằng lệnh

120

Mô hình ứng dụng Client-Server sử dụng Socket ở chế độ có nối kết (TCP) (3/3)



121

Lập trình socket với java

Java hỗ trợ lập trình mạng thông qua các lớp trong gói java.net.

Một số lớp tiêu biểu được dùng cho lập trình Client-Server sử dụng socket làm phương tiện giao tiếp như:

- **InetAddress**: Lớp này quản lý địa chỉ Internet bao gồm địa chỉ IP và tên máy tính.
- **Socket**: Hỗ trợ các phương thức liên quan đến Socket cho chương trình Client ở chế độ có nối kết.
- **ServerSocket**: Hỗ trợ các phương thức liên quan đến Socket cho chương trình Server ở chế độ có nối kết.
- **DatagramSocket**: Hỗ trợ các phương thức liên quan đến Socket ở chế độ không nối kết cho cả Client và Server.
- **DatagramPacket**: Lớp cài đặt gói tin dạng thư tín người dùng (Datagram Packet) trong giao tiếp giữa Client và Server ở chế độ không nối kết.

123

Xây dựng chương trình Client ở chế độ có nối kết

Các bước tổng quát:

1. Mở một socket nối kết đến server đã biết địa chỉ IP (hay tên miền) và số hiệu cổng.
2. *Lấy **InputStream** và **OutputStream** gán với **Socket**.*
3. Tham khảo Protocol của dịch vụ để định dạng đúng dữ liệu trao đổi với Server.
4. Trao đổi dữ liệu với Server nhờ vào các **InputStream** và **OutputStream**.
5. Đóng Socket trước khi kết thúc chương trình.

124

Lớp java.net.Socket (1/2)

Lớp Socket hỗ trợ các phương thức cần thiết để xây dựng các chương trình client sử dụng socket ở chế độ có nối kết. Các phương thức:

- public Socket(String HostName, int PortNumber) throws IOException

dùng để nối kết đến một server có tên là HostName, cổng là PortNumber, kết quả là một kênh ảo được thành lập giữa client và server

•HostName: Địa chỉ IP hoặc tên logic theo dạng tên miền.

•PortNumber: có giá trị từ 0 ..65535 Ví

dụ:

```
Socket s = new Socket("www.ptithcm.edu.vn",80);
```

```
Hoặc: Socket s = new Socket("113.161.98.131",80);
```

125

Lớp java.net.Socket (2/2)

- public InputStream getInputStream()

Trả về InputStream nối với Socket, chương trình Client dùng InputStream này để nhận dữ liệu từ Server gửi về.

Ví dụ: Lấy InputStream của Socket s:

```
InputStream is = s.getInputStream();
```

- public OutputStream getOutputStream()

Trả về OutputStream nối với Socket. Chương trình Client dùng OutputStream này để gửi dữ liệu cho Server.

Ví dụ: Lấy OutputStream của Socket s:

```
OutputStream os = s.getOutputStream();
```

- public close()

Đóng Socket lại, giải phóng kênh ảo, xóa nối kết giữa Client và Server.

126

Ví dụ chương trình TCPEchoClient

```
import java.io.*;
import java.net.Socket;
public class TCPEchoClient {
    public final static String serverIP = "127.0.0.1";
    public final static int serverPort = 7;
    Run | Debug
    public static void main(String[] args) throws InterruptedException, IOException {
        Socket s = null;
        try {
            s = new Socket(serverIP, serverPort);
            System.out.println("Client da duoc tao");
            InputStream is = s.getInputStream();
            OutputStream os = s.getOutputStream();
            for(int i='0'; i<='9'; i++) { //goi '0'-'9' den server
                os.write(i);
                int ch = is.read();
                System.out.println((char)ch);
                Thread.sleep(2000);
            }
        } catch (IOException ie) {
            System.out.println("Error: Can NOT create socket");
        } finally {
            if (s != null) {
                s.close();
            }
        }
    }
}
```

127

Xây dựng chương trình Server ở chế độ có nối kết

Có hai trường hợp:

- Server phục vụ theo kiểu tuần tự
- Server phục vụ theo kiểu song song

128

Lớp java.net.ServerSocket

Lớp ServerSocket hỗ trợ các phương thức cần thiết để xây dựng các chương trình Server sử dụng socket ở chế độ có nối kết. Có các phương thức quan trọng:

-public ServerSocket(int PortNumber);

Tạo một Socket với số hiệu cổng là PortNumber mà sau đó Server sẽ lắng nghe trên cổng này.

Ví dụ: Tạo socket cho Server với số hiệu cổng là

```
7: ServerSocket ss = new ServerSocket(7);
```

-public Socket accept();

Lắng nghe yêu cầu nối kết của các Client. Hoạt động ở chế độ ngẩn, sẽ bị ngẩn cho đến khi có một yêu cầu nối kết của client gửi đến.

Khi có yêu cầu nối kết của Client gửi đến, nó sẽ chấp nhận yêu cầu nối kết, trả về một Socket là một đầu của kênh giao tiếp ảo giữa Server và Client yêu cầu nối kết.

Ví dụ: Socket ss chờ nhận yêu cầu nối

```
kết: Socket s = ss.accept();
```

Server sau đó sẽ lấy InputStream và OutputStream của Socket mới s để giao tiếp với Client.

129

Xây dựng chương trình Server phục vụ tuần tự

-Tại một thời điểm Server chỉ chấp nhận một yêu cầu nối kết. Các yêu cầu nối kết của các Client khác đều không được đáp ứng.

Các bước tổng quát của một Server phục vụ tuần tự

1. Tạo socket và gán số hiệu cổng cho server.
2. Lắng nghe yêu cầu nối kết.
3. Với một yêu cầu nối kết được chấp nhận thực hiện các bước sau:
 - Lấy InputStream và OutputStream gắn với Socket của kênh ảo vừa được hình thành.
 - Lặp lại công việc sau:
 - Chờ nhận các yêu cầu (công việc).
 - Phân tích và thực hiện yêu cầu.
 - Tạo thông điệp trả lời.
 - Gửi thông điệp trả lời về Client.
 - Nếu không còn yêu cầu hoặc Client kết thúc, đóng Socket và quay lại bước 2.

130

```
import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;

public class TCPEchoServer {
    public final static int serverPort = 7;
    Run | Debug
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(serverPort);
            System.out.println("Server da duoc tao");
            while(true){
                try {
                    Socket s = ss.accept();
                    OutputStream os = s.getOutputStream();
                    InputStream is = s.getInputStream();
                    int ch = 0;
                    while(true) {
                        ch = is.read();
                        if(ch == -1) break;
                        System.out.println((char)ch);
                        os.write(ch);
                    }
                    s.close();
                } catch (IOException ie1) {
                    System.out.println("Connection Error: "+ie1);
                }
            }
        } catch (IOException ie) {
            System.out.println("Server Creation Error: "+ie);
        }
    }
}
```

Ví dụ
Chương trình
STCPEchoServer

131

Xây dựng chương trình Server phục vụ song song

- Tại một thời điểm Server chấp nhận nhiều yêu cầu nối kết và phục vụ nhiều Client cùng lúc.
- Chương trình Server phục vụ song song gồm 2 phần thực hiện song song nhau:
 - Phần 1: Xử lý các yêu cầu nối kết.
 - Phần 2: Xử lý các thông điệp yêu cầu từ khách hàng.

Phần 1: Lặp lại các công việc sau:

- Lắng nghe yêu cầu kết nối của khách hàng.
- Chấp nhận một yêu cầu kết nối.
Tạo kênh giao tiếp ảo mới với khách hàng.
- Tạo Phần 2 để xử lý các thông điệp yêu cầu của khách hàng.

Phần 2: Lặp lại các công việc sau:

- Chờ nhận thông điệp yêu cầu của khách hàng.
- Phân tích và xử lý yêu cầu.
- Gửi thông điệp trả lời cho khách hàng.

Ứng với mỗi client trên server có một phần 2, phần 2 kết thúc khi kênh ảo bị xóa

□ thiết kế theo thread, phần 1 là phân phối luôn tồn tại trên server, phần 2 là các thread công tác (worked thread)

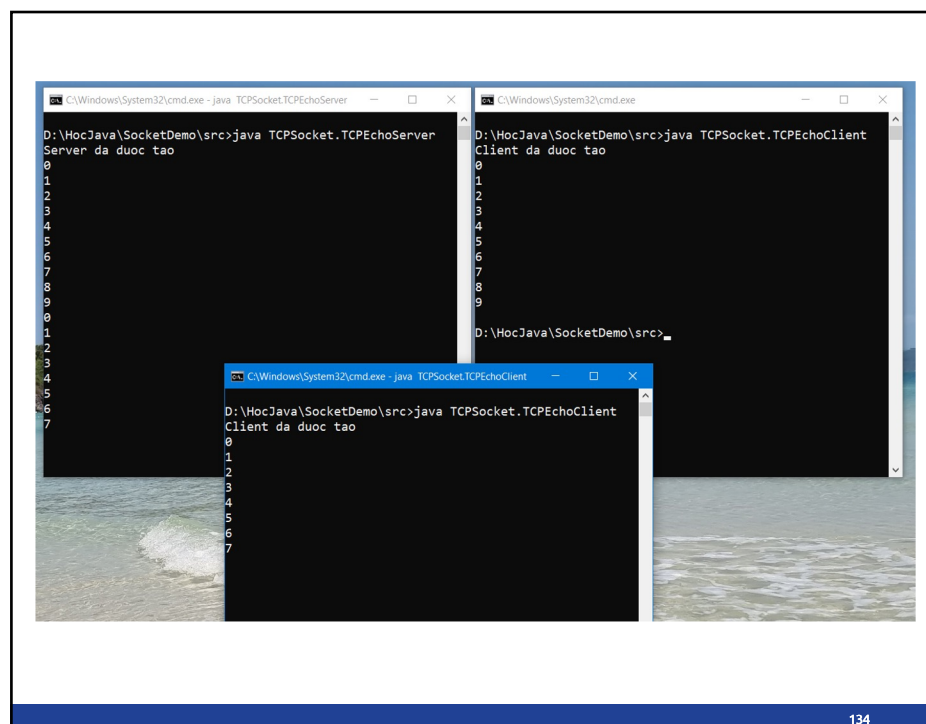
132

Ví dụ chương trình TCPEchoServer

```
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;

public class RequestProcessing extends Thread{
    Socket channel; //socket của kênh ao noi voi clien hien tai
    public RequestProcessing(Socket s) {
        channel = s;    //nhan socket của kênh ao noi voi client
    }
    public void run() {
        try{
            OutputStream os = channel.getOutputStream();
            InputStream is = channel.getInputStream();
            while(true) {
                int n = is.read();
                if(n==-1) break;
                os.write(n);
                //
            }
        }catch(IOException ie) {
            System.out.println("Request Processing Error: "+ ie);
        }
    }
}
```

133



134

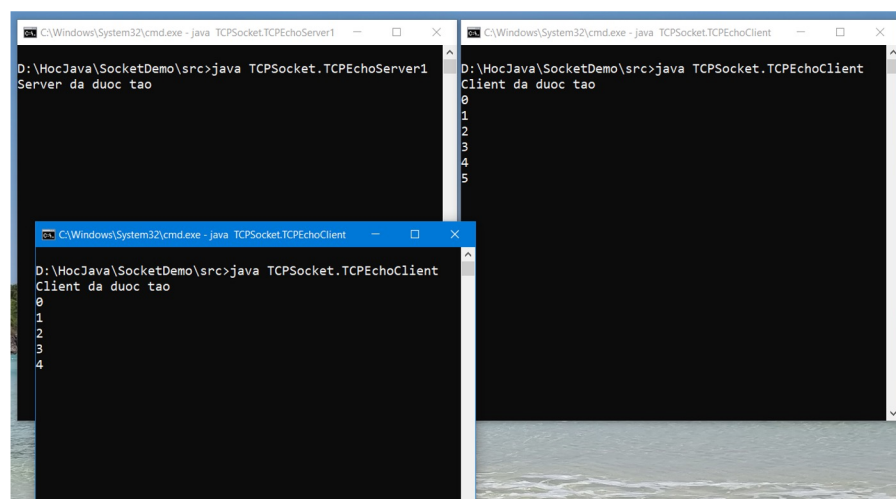
134

```
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.ServerSocket;
import java.net.Socket;
```

```
public class TCPEchoServer1 {
    public final static int serverPort = 7;
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(serverPort);
            System.out.println("Server da duoc tao");
            while(true){
                try {
                    Socket s = ss.accept();
                    RequestProcessing rp = new RequestProcessing(s);
                    rp.start();
                } catch(IOException ie1) {
                    System.out.println("Connection Error: "+ie1);
                }
            }
        } catch(IOException ie) {
            System.out.println("Server Creation Error: "+ie);
        }
    }
}
```

Ví dụ
chương trình
PTCPEchoServer
(tt)

135



136

136

Xây dựng ứng dụng Client/Server ở chế độ không nối kết

- Mô hình client - server sử dụng lớp `ServerSocket` và `Socket` ở trên dùng giao thức UDP.
- Nếu muốn sử dụng mô hình client - server với giao thức UDP, sử dụng hai lớp **`java.net.DatagramSocket`** và **`java.net.DatagramPacket`**.
- **`DatagramSocket`** được sử dụng để truyền và nhận các **`DatagramPacket`**.
- Dữ liệu được truyền đi là **một mảng những byte**, chúng được **gói vào trong lớp `DatagramPacket`**

137

Lớp `DatagramPacket`

`public DatagramPacket(byte[] b, int n)`

Tạo ra một `DatagramPacket` chứa **`n` bytes dữ liệu đầu tiên của mảng `b`**.

Trả về một đối tượng thuộc lớp `DatagramPacket`

Ví dụ: Tạo `DatagramPacket` để nhận dữ liệu:

```
byte buff[] = new byte[60000];
DatagramPacket inPacket = new DatagramPacket(buff, buff.length);
```

`public DatagramPacket(byte[] b, int n, InetAddress ia, int port)`

Tạo một `DatagramPacket` chứa dữ liệu và cả địa chỉ của máy nhận dữ liệu

Ví dụ:

```
try {
    //Địa chỉ Internet của máy nhận
    InetAddress ia = InetAddress.getByName("www.iuh.edu.vn");
    int port = 80; // Cổng của socket nhận
    String s = "My second UDP Packet"; // Dữ liệu gửi đi
    byte[] b = s.getBytes(); // Chuyển chuỗi thành mảng bytes
    // Tạo gói tin gửi đi
    DatagramPacket outPacket = new DatagramPacket(b, b.length, ia, port);
} catch (UnknownHostException e) {System.err.println(e); }
```

138

Lớp DatagramPacket (tt)

Các phương thức lấy thông tin trên một DatagramPacket nhận được

Khi nhận được một DatagramPacket từ một quá trình khác gửi đến, ta có thể lấy thông tin trên

DatagramPacket này bằng các phương thức sau:

- public synchronized() InetAddress getAddress() : Địa chỉ máy gửi
- public synchronized() int getPort() : Cổng của quá trình gửi
- public synchronized() byte[] getData() : Dữ liệu từ gói tin
- public synchronized() int getLength() : Chiều dài của dữ liệu trong gói tin

139

Lớp DatagramPacket (tt)

Các phương thức đặt thông tin cho gói tin gửi

Trước khi gửi một DatagramPacket đi, ta có thể đặt thông tin trên DatagramPacket này bằng các phương thức sau:

- public synchronized() void setAddress(InetAddress dis) : Đặt địa chỉ máy nhận.
- public synchronized() void setPort(int port) : Đặt cổng quá trình nhận
- public synchronized() void setData(byte buffer[]) : Đặt dữ liệu gửi
- public synchronized() void setLength(int len) : Đặt chiều dài dữ liệu gửi

140

Lớp DatagramSocket

public DatagramSocket() throws SocketException

- Tạo Socket kiểu không nối kết cho Client. Hệ thống tự động gán số hiệu cổng chưa sử dụng cho socket.

- Ví dụ: Tạo một socket không nối kết cho Client:

```
try {
    DatagramSocket ds = new DatagramSocket();
} catch(SocketException se) {
    System.out.print("Create DatagramSocket
Error:"+se);
}
```

141

Lớp DatagramSocket (tt)

public DatagramSocket(int port) throws SocketException

- Tạo Socket kiểu không nối kết cho Server với số hiệu cổng được xác định trong tham số (port).

- Ví dụ: Tạo một socket không nối kết cho Server với số hiệu cổng là 7:

```
try {
    DatagramSocket dp = new DatagramSocket(7);
} catch(SocketException se) {
    System.out.print("Create DatagramSocket Error: "+se);
}
```

142

Lớp DatagramSocket (tt)

`public void send(DatagramPacket dp) throws IOException`

- Dùng để gửi một DatagramPacket đi.
- Ví dụ: Gửi chuỗi "My second UDP Packet" cho quá trình ở địa chỉ www.sgu.edu.vn, cổng nhận là 19:


```
try {
    DatagramSocket ds = new DatagramSocket(); //Tạo Socket
    //Địa chỉ Internet của máy nhận
    InetAddress ia = InetAddress.getByName("www.sgu.edu.vn");
    int port = 19; // Cổng của quá trình nhận
    String s = "My second UDP Packet"; // Dữ liệu cần gửi
    byte[] b = s.getBytes(); // Đổi sang mảng bytes
    // Tạo gói tin
    DatagramPacket outPacket = new DatagramPacket(b, b.length, ia, port);
    ds.send(outPacket); // Gửi gói tin đi
}
catch (IOException e) { System.err.println(e); }
```

143

Lớp DatagramSocket (tt)

`public synchronized void receive(DatagramPacket dp) throws IOException`

- Chờ nhận một DatagramPacket. Quá trình sẽ bị nghẽn cho đến khi có dữ liệu đến.**
- Ví dụ:


```
try {
    DatagramSocket ds = new DatagramSocket(); //Tạo
    Socket byte[] b = new byte[60000]; // Nơi chứa dữ liệu nhận được
    DatagramPacket inPacket = new DatagramPacket(b, b.length); // Tạo gói tin ds.receive(inPacket); // Chờ
    nhận gói tin
}
catch (IOException e) {
    System.err.println(e);
}
```

144

```

import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;

public class UDPEchoServer {

    public final static int serverPort = 7;

    public static void main(String[] args) {
        try {
            DatagramSocket ds = new DatagramSocket(serverPort);
            System.out.println("Server created...");
            byte[] buffer = new byte[6000];
            while(true) {
                DatagramPacket incoming = new DatagramPacket(buffer, buffer.length);
                ds.receive(incoming); //cho'nhân gói tin gói đến
                //lay du lieu khoi gói tin nhan va doi tu mang byte ra chuỗi
                String theString = new String(incoming.getData(),0,incoming.getLength());
                //tao gói tin chua du lieu vua nhan duoc
                DatagramPacket outsendng = new DatagramPacket(theString.getBytes(),
                    incoming.getLength(), incoming.getAddress(), incoming.getPort());
                ds.send(outsendng);
            }
        } catch (IOException ie) {
            System.out.println(ie);
        }
    }
}

```

Run | Debug

Chương trình UDPEchoServer

145

```

public class UDPEchoClient {
    public final static int serverPort = 7;

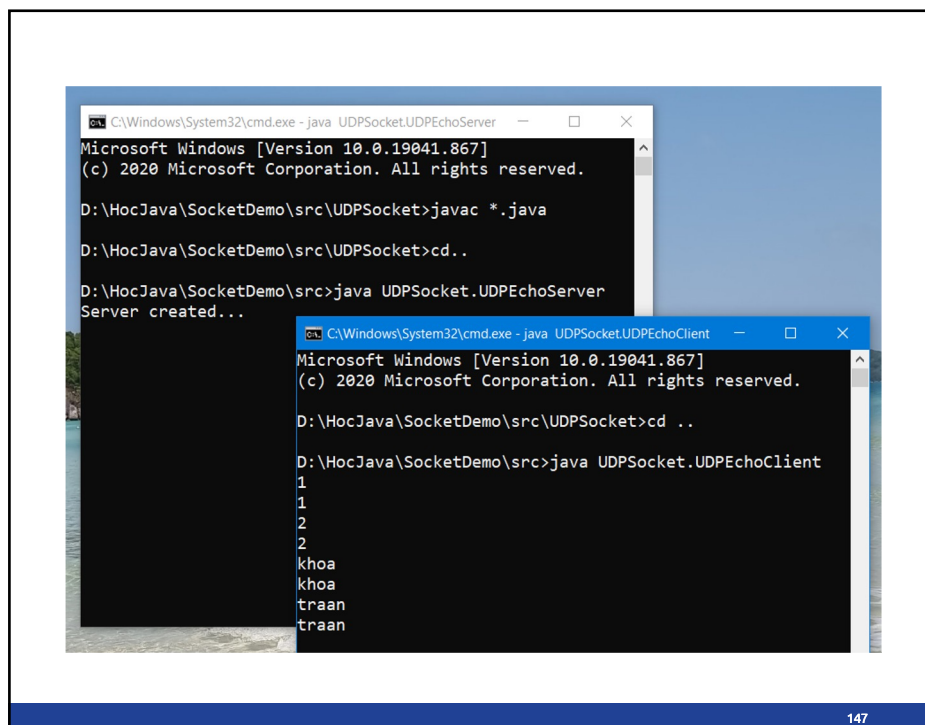
    public static void main(String[] args) {
        try {
            if(args.Length == 0) { //kiem tra tham so, la dia chi cua server
                System.out.println("Syntax: java UDPClient Hostname");
                return;
            }
            DatagramSocket ds = new DatagramSocket();
            InetAddress server = InetAddress.getByName("localhost");
            while(true) {
                InputStreamReader isr = new InputStreamReader(System.in);
                BufferedReader br = new BufferedReader(isr);
                String theString = br.readLine();
                byte[] data = theString.getBytes(); //doi chuỗi ra mảng byte
                //tao gói tin gói
                DatagramPacket dp = new DatagramPacket(data, data.length, server, serverPort);
                ds.send(dp);
                byte[] buffer = new byte[6000]; //vung dem cho du lieu nhan
                //gói tin nhan
                DatagramPacket incoming = new DatagramPacket(buffer, buffer.length);
                ds.receive(incoming);
                //doi du lieu nhan duoc dang mảng bytes sang chuỗi va in ra màn hình
                System.out.println(new String(incoming.getData(), 0, incoming.getLength()));
            }
        } catch (IOException ie) {
            System.out.println(ie);
        }
    }
}

```

Run | Debug

Chương trình UDPEchoClient

146



147

Bài tập

Bài 1. Viết lại các chương trình ví dụ ở chế độ có kết nối, sao cho có thể xuất nhập với chuỗi.

Bài 2. Viết chương trình nhận đối số là một URL. Nối kết đến Web Server trong URL nhận được, lấy trang web về và in ra màn hình theo dạng textfile (html).

Bài 3. Viết chương trình theo mô hình Client-Server sử dụng dụng Socket ở chế độ có nối kết. Trong đó :

+ Server làm nhiệm vụ đọc một ký tự số từ '0' đến '9'.

(Ví dụ : nhận số 0 : trả về "khong" , 1 : trả về "một" ; 9 : trả về "chín", nếu nhận ký tự khác số thì trả về "Không phải số nguyên").

+ Client sẽ nhập vào 1 ký tự, gửi qua Server, nhận kết quả trả về từ Server và thể hiện lên màn hình

148

- Broadcast (255.255.255.255)
- Multicast (224.0.0.0-239.255.255.255) group (join/leave)
- Unicast (1-1)

149

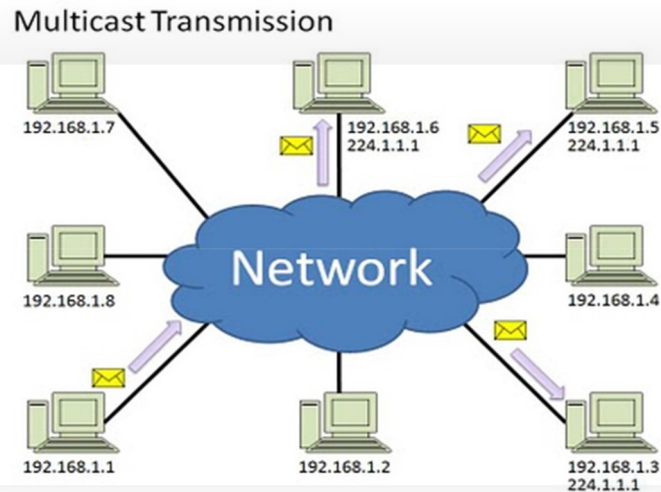
149

3. LẬP TRÌNH MULTICAST



150

Multicast networking



151

Multicast networking

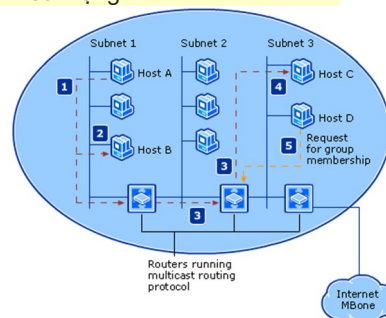
- TTL - Time To Live: Số router tối đa mà một gói multicast có thể đi qua. (==thời gian sống của multicast packet tính từ khi nó được gửi đi đến khi được hủy bởi các router)

TTL=0 : packet không được gửi

TTL=1 (mặc định) : packet chỉ gửi trong mạng cục bộ.

TTL>1: packet có thể gửi qua nhiều mạng khác nhau

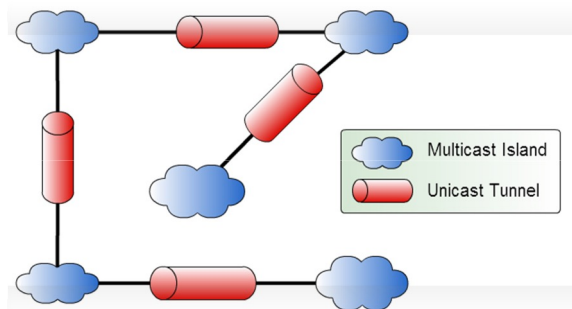
- Broadcast: Gửi packet đến tất cả các IP hosts trong một subnet
- Multicast: packet có thể đi ngang qua nhiều mạng



152

MBone

- MBone = Internet Multicast Backbone
- Largest deployment of multicast on the Internet
- Multicast island: network or group of adjacent networks that supports multicast
- TCP/IP tunnelling: scheme of moving multicast packets by putting them in regular unicast IP packets
- TTL ≤ 64 restricts a packet to the local multicast island



153

Importance & limitations of multicast

- **Importance**
 - Reduce transmission traffic
 - Reduce load on network servers
- **Limitations**
 - Routers & switches need to be multicast-enabled
 - Need management tools for multicast deployment
 - Complete deployment of IP multicast over Internet is very difficult
 - No security measures to protect multicast traffic now, e.g. cannot restrict group joining

154

Địa chỉ Multicast

- Miêu tả trong RFC 1112
- Sử dụng IGMP (Internet Group Multicasting Protocol)
- Sử dụng lớp D của địa chỉ IP: từ 224.0.0.0 đến 239.255.255.255.
Tuy nhiên địa chỉ 224.0.0.0 được để lại và không nên dùng.
- Mỗi địa chỉ IP trong khoảng này biểu diễn cho một nhóm multicast
- Một địa chỉ IP trong nhóm multicast được sử dụng chung cho tất cả các thành viên của nhóm để gửi và nhận dữ liệu.

155

155

Lập trình Multicast

- Phải sử dụng giao thức UDP
- Các bước lập trình multicast:
 1. Tạo một UDP socket
 2. Tham gia một nhóm multicast chỉ ra bởi một địa chỉ IP lớp D
 3. Nhận các packet gửi đến cho nhóm đó
 4. Gửi các packet đến các máy trong nhóm thông qua địa chỉ IP của nhóm.
 5. Rời bỏ khỏi nhóm multicast
 6. Đóng socket

156

156

Một số vấn đề của Multicast

- Nhóm động: các host có thể tham gia và rời bỏ nhóm bất kỳ thời điểm nào
- Một host có thể tham gia một nhóm multicast bằng cách gửi một IGMP message
- Một nhóm có số thành viên không giới hạn
- Một máy có thể gửi một message cho một nhóm mà không cần tham gia vào nhóm đó.

157

157

Time-To-Live (TTL)

- TTL là thời gian sống của multicast packet tính từ khi nó được gửi đi đến khi được hủy bởi các router.
- TTL=0 : packet không được gửi
- TTL=1 (mặc định) : packet chỉ gửi trong mạng cục bộ.
- TTL>1: packet có thể gửi qua nhiều mạng khác nhau

158

158

Multicast trong Java

- **MulticastSocket** là lớp con của **DatagramSocket**.
- Được dùng để truyền và nhận các gói IP multicast.

161

Constructors

```
MulticastSocket()
```

```
MulticastSocket(int port)
```

```
MulticastSocket(SocketAddress bindaddr)
```

162

Chi tiết hàm tạo (1/3)

public MmmMuuuuullltttiiiccccaaaassssttttSSSSoooocccckkkkeeeett() throws [IOException](#).

Create a multicast socket.

- When the socket is created the [DatagramSocket.setReuseAddress\(boolean\)](#) method is called to enable the SO_REUSEADDR socket option.

Throws:

[IOException](#) - if an I/O exception occurs while creating the MulticastSocket

163

Chi tiết hàm tạo (2/3)

public MmmMuuuuullltttiiiccccaaaassssttttSSSSoooocccckkkkeeeett(int port) throws [IOException](#).

Create a multicast socket and bind it to a specific port.

- When the socket is created the [DatagramSocket.setReuseAddress\(boolean\)](#) method is called to enable the SO_REUSEADDR socket option.

Parameters: port -

port to use **Throws:**

[IOException](#) - if an I/O exception occurs while creating the MulticastSocket

164

Chi tiết hàm tạo (3/3)

```
public MmmMuuuullllttttiiiccccaaasssstttSSSSoooocccckkkkeeeetttt(SocketAddress bindaddr) throws IOException
```

Create a MulticastSocket bound to the specified socket address. Or, if the address is null, create an unbound socket.

- When the socket is created the [DatagramSocket.setReuseAddress\(boolean\)](#) method is called to enable the SO_REUSEADDR socket option.

Parameters: bindaddr - Socket address to bind to, or null for an unbound socket.

Throws:
[IOException](#) - if an I/O exception occurs while creating the MulticastSocket

165

setTimeToLive

```
public void sssseeeetttTTTTiiiiimmmeeeeTTTTTooooLLLLiiiiivvvveeee(int ttl) throws IOException
```

- Set the default time-to-live for multicast packets sent out on this MulticastSocket in order to control the scope of the multicasts.
- The ttl **must** be in the range $0 \leq \text{ttl} \leq 255$ or an IllegalArgumentException will be thrown.

Parameters:

ttl - the time-to-live

Throws:

[IOException](#) - if an I/O exception occurs while setting the default time-to-live value

166

getTimeToLive

```
public int ggggeeeetttTTTTiiiiimmmmeeeeTTTTooooLLLLiiiiivvvveeee() throws IOException
```

Get the default time-to-live for multicast packets sent out on the socket.

Returns:

the default time-to-live value

Throws:

[IOException](#) - if an I/O exception occurs while getting the default time-to-live value

167

joinGroup

```
public void jjjjoooliiiiinnnnGGGGrrrrrooooouuupppp(InetAddress mcastaddr) throws IOException
```

- Joins a multicast group. Its behavior may be affected by `setInterface` or `setNetworkInterface`.

Parameters:

`mcastaddr` - is the multicast address to join

Throws:

[IOException](#) - if there is an error joining or when the address is not a multicast address.

168

Dạng khác của joinGroup

```
public void jJJJoOoOiiInnnnGGGGrrrrrooooouuuppp(SocketAddress mcastaddr,
NetworkInterface netIf) throws IOException
```

- Joins the specified multicast group at the specified interface..

Parameters:

mcastaddr - is the multicast address to join

netIf - specifies the local interface to receive multicast datagram packets, or *null* to defer to the interface set by

[setInterface\(InetAddress\)](#) or [setNetworkInterface\(NetworkInterface\)](#)

169

leaveGroup

```
public void lllleeeaaaavvvveeeGGGGrrrrrooooouuuppp(InetAddress
mcastaddr) throws IOException
```

- Leave a multicast group. Its behavior may be affected by
setInterface or setNetworkInterface.

Parameters:

mcastaddr - is the multicast address to leave

Throws:

[IOException](#) - if there is an error leaving or when the address is not a multicast address.

170

Dạng khác của leaveGroup

```
public void lllleeeeeaaaavvvveeeeGGGGrrrrrooooouuuppp(SocketAddress mcastaddr,
NetworkInterface netIf) throws IOException
```

Leave a multicast group on a specified local interface.

Parameters:

mcastaddr - is the multicast address to leave

netIf - specifies the local interface or *null* to defer to the interface set

by [setInterface\(InetAddress\)](#) or [setNetworkInterface\(NetworkInterface\)](#).

Throws:

[IOException](#) - if there is an error leaving or when the address is not a multicast address.

171

setInterface

```
public void sssseeeettllllnnnnntttteerrrrrrfffaaaacccceeee(InetAddress inf) throws
SocketException
```

Set the multicast network interface used by methods whose behavior would be affected by the value of the network interface. Useful for multihomed hosts.

Parameters:

inf : the InetAddress

Throws:

[SocketException](#) - if there is an error in the underlying protocol, such as a TCP error.

172

getInterface

```
public InetAddress ggggeeeetttllllnnnnntttteerrrrrrfffaaaacccceeee() throws SocketException
```

Retrieve the address of the network interface used for multicast packets.

Returns:

An [InetAddress](#) representing the address of the network interface used for multicast packets.

Throws:

[SocketException](#) - if there is an error in the underlying protocol, such as a TCP error.

173

setNetworkInterface

```
public void  
ssssseeeetttNNNNeeeeeetttwwwwoooooorrrkkkkllllnnnnntttteerrrrrrfffaaaacccceeee(NetworkInterface netIf)  
throws SocketException
```

Specify the network interface for outgoing multicast datagrams sent on this socket.

Parameters:

netIf - the interface

Throws:

[SocketException](#) - if there is an error in the underlying protocol, such as a TCP error.

174

getNetworkInterface

```
public NetworkInterface
gggggeeeettttNNNNeeeeeettttwwwoooooorrrrrkkkkllllnnnnntttteeeerrrrfffaaaacccceeee() throws
SocketException
```

Get the multicast network interface set.

Returns:

the multicast `NetworkInterface` currently set

Throws:

[SocketException](#) - if there is an error in the underlying protocol, such as a TCP error.

175

setLoopbackMode

```
public void
ssssseeeettttLLLLooooooooppbbbbbbaaaacccckkkkMMMMooooddddeeee(boolean disable)
throws SocketException
```

Disable/Enable local loopback of multicast datagrams

The option is used by the platform's networking code as a hint for setting whether multicast data will be looped back to the local socket.

Because this option is a hint, applications that want to verify what loopback mode is set to should call [getLoopbackMode\(\)](#)

Parameters:

disable - true to disable the LoopbackMode

Throws:

[SocketException](#) - if an error occurs while setting the value

176

getLoopbackMode

```
public boolean ggggeeeetttLLLoooooooooppbbbaaaacccckkkkMMMoooooddddeeee()
throws SocketException
```

Get the setting for local loopback of multicast datagrams.

Returns:

true if the LoopbackMode has been disabled

Throws:

SocketException - if an error occurs while getting the value

177

sen

@Deprecated

```
public void sssseeeennnnndddd(DatagramPacket p, byte ttl) throws IOException
```

Sends a datagram packet to the destination, with a TTL (time- to-live) other than the default for the socket. Its behavior may be affected by `setInterface..`

Parameters:

p - is the packet to be sent. The packet should contain the destination multicast ip address and the data to be sent. One does not need to be the member of the group to send packets to a destination multicast address.

t_{tl} - optional time to live for multicast packet. default t_{tl} is 1.

Throws:

IOException - is raised if an error occurs i.e error while setting ttl.

178

Các Method thừa kế từ java.net. DatagramSocket

```
bind, close, connect, connect, disconnect, getBroadcast, getChannel,
getInetAddress, getLocalAddress, getLocalPort, getLocalSocketAddress,
getPort, getReceiveBufferSize, getRemoteSocketAddress,
getReuseAddress, getSendBufferSize, getSoTimeout,
getTrafficClass, isBound, isClosed, isConnected, receive, send,
setBroadcast, setDatagramSocketImplFactory, setReceiveBufferSize,
setReuseAddress, setSendBufferSize,
setSoTimeout, setTrafficClass
```

179

Lập trình multicast

- Tương tự như dùng DatagramSocket ngoại trừ:
 - Khi gửi multicast packet có thể chỉ giá trị TTL
 - Để nhận multicast packet trước hết phải gia nhập một nhóm multicast
- Nhiều MulticastSocket có thể lắng nghe trên cùng một port của cùng một máy (DatagramSocket thì không thể)
- Không thể tạo MulticastSocket trên một UDP port đang được dùng bởi một DatagramSocket trên cùng một host

180

Ví dụ 1

```

import java.net.*;
import java.io.*;
import java.util.*;

public class DaytimeMulticastServer {

    public static final int defaultPort = 1234;

    public static void main (String args[] throws Exception {
        if (args.length > 1 ) throw new IllegalArgumentException
            ("Syntax: DaytimeMulticastServer [<multicastgroup>"]);
        InetAddress multicastGroup = InetAddress.getByName(
            args.length==0 ? "234.5.6.7": args[0]);
        MulticastSocket socket = new MulticastSocket();
        // socket.joinGroup(multicastGroup);
        while (true) {
            Thread.sleep(5000);
            System.out.println("Multicast Data ...");
            String str = (new Date()).toString();
            byte[] data = str.getBytes();
            DatagramPacket packet = new DatagramPacket(
                data, data.length, multicastGroup, defaultPort);
            socket.send(packet);
        }
    }
}

```

181

181

```

import java.net.*;
import java.io.*;
import java.util.*;

public class DaytimeMulticastClient {

    public static final int defaultPort = 1234;

    public static void main (String args[] throws Exception {
        if (args.length > 1 ) throw new IllegalArgumentException
            ("Syntax: DaytimeMulticastClient [<multicastgroup>"]);
        InetAddress multicastGroup = InetAddress.getByName(
            args.length==0 ? "234.5.6.7": args[0]);
        MulticastSocket socket = new MulticastSocket(defaultPort);
        socket.joinGroup(multicastGroup);
        byte[] buffer = new byte[80];
        DatagramPacket packet = new DatagramPacket (buffer,
buffer.length);
        socket.receive(packet);
        String str = new String(packet.getData());
        System.out.println("Time received from
" + packet.getAddress() + " is: " + str);
    }
}

```

182

182

Ví dụ2

```
import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.rmi.UnknownHostException;

public class MulticastSocketServer {
    final static String INET_ADDR = "224.0.0.3";
    final static int PORT = 8888;

    Run | Debug
    public static void main(String[] args) throws UnknownHostException, InterruptedException {

        try (DatagramSocket serverSocket = new DatagramSocket()){
            InetAddress addr = InetAddress.getByName(INET_ADDR);
            for(int i=0; i<10; i++){
                String msg = "Sent msg no." + i;
                DatagramPacket msgPacket = new DatagramPacket(msg.getBytes(),
                                                                msg.getBytes().length, addr, PORT);
                serverSocket.send(msgPacket);

                System.out.println("Server sent packet with msg: " + msg);
                Thread.sleep(1000);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

183

```
import java.net.DatagramPacket;
import java.net.InetAddress;
import java.net.MulticastSocket;
import java.net.UnknownHostException;

public class MulticastSocketClient {
    final static String INET_ADDR = "224.0.0.3";
    final static int PORT = 8888;

    Run | Debug
    public static void main(String[] args) throws UnknownHostException{
        InetAddress address = InetAddress.getByName(INET_ADDR);

        byte[] buff = new byte[256];

        try (MulticastSocket clientSocket = new MulticastSocket(PORT)){
            clientSocket.joinGroup(address);

            while(true){
                DatagramPacket msgPacket = new DatagramPacket(buff, buff.length);
                clientSocket.receive(msgPacket);

                String msg = new String(buff, 0, buff.length);
                System.out.println("Socket 1 received msg: " + msg);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

184

```

D:\HocJava\SocketDemo\src\MulticastSocket>java MulticastS
ocketServer
Server sent packet with msg: Sent msg no.0
Server sent packet with msg: Sent msg no.1
Server sent packet with msg: Sent msg no.2
Server sent packet with msg: Sent msg no.3
Server sent packet with msg: Sent msg no.4
Server sent packet with msg: Sent msg no.5
Server sent packet with msg: Sent msg no.6
Server sent packet with msg: Sent msg no.7
Server sent packet with msg: Sent msg no.8
Server sent packet with msg: Sent msg no.9
D:\HocJava\SocketDemo\src\MulticastSocket>

D:\HocJava\SocketDemo\src\MulticastSocket>java MulticastS
ocketClient
Socket 1 received msg: Sent msg no.3
Socket 1 received msg: Sent msg no.4
Socket 1 received msg: Sent msg no.5
Socket 1 received msg: Sent msg no.6
Socket 1 received msg: Sent msg no.7
Socket 1 received msg: Sent msg no.8

D:\HocJava\SocketDemo\src\MulticastSocket>java MulticastS
ocketClient2
Socket 1 received msg: Sent msg no.0
Socket 1 received msg: Sent msg no.1
Socket 1 received msg: Sent msg no.2
Socket 1 received msg: Sent msg no.3

```

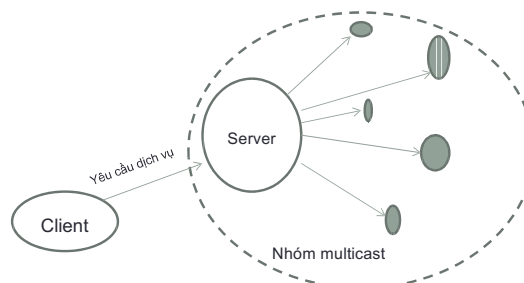
185

185

Bài tập

Thiết kế và lập trình ứng dụng dịch vụ multicast như sau:

- Server đóng vai trò là trung tâm phục vụ quảng bá thông tin cho các client. Server hoạt động theo chế độ tuần tự.
- Mỗi khi muốn quảng bá thông tin đến nhóm multicast, các client sẽ gửi thông tin đến server và nhờ server quảng bá đến nhóm multicast



186